Sviluppo di un programma

Scrittura e compilazione

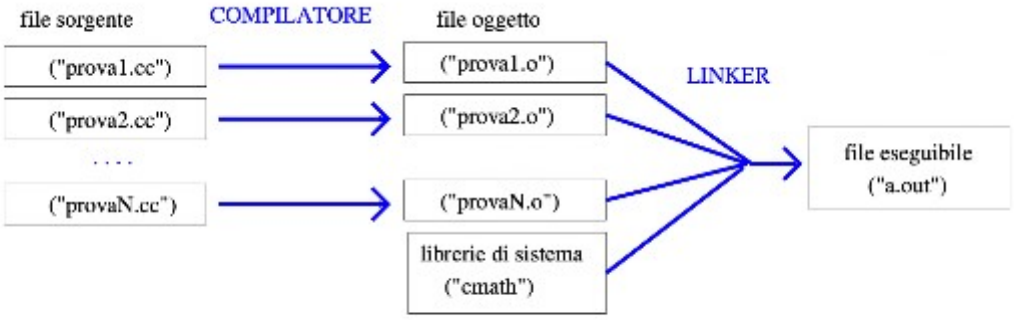
Scrittura di uno o più file di testo chiamati sorgenti



- File sorgente tradotto in un file oggetto dal compilatore
  - Es: `g++ -c prova.cc`
  - Imp: file oggetto illeggibile ad un essere umano e non stampabile!
- File oggetto collegato (linked) a librerie di sistema dal linker, generano un file eseguibile (default `a.out`, opzione `-o <nome>`)
  - Es: `g++ prova.o`
  - Es: `g++ prova.o -o prova`
  - File incomprensibili agli umani, ma eseguibili da una macchina
- Compilazione e linking possibile in un unica istruzione
  - Es: `g++ prova.cc`

Il processo di compilazione consiste nel tradurre il file sorgente in un file oggetto (questa operazione la fa il compilatore), successivamente bisogna generare il file eseguibile collegando il file oggetto con le librerie di sistema (questa operazione la fa il linker).

La stessa cosa vale nel caso di più file sorgente:



- File sorgente tradotti nei rispettivi file oggetto uno alla volta
- File oggetto collegato (linked) a librerie di sistema dal linker, generano un file eseguibile (default `a.out`)
  - Es: `g++ prova1.o prova2.o ... provaN.o`
- Compilazione e linking possibile in un unica istruzione
  - Es: `g++ prova1.cc prova2.cc ... provaN.cc`

L'istruzione -o consente di dare un nome diverso all'eseguibile (di default è "a")

Elementi di base

Le parole di un programma C++ possono essere costituite da:

- Lettere:
  `_ a b c d e f g h i j k l m n o p q r s t u v w x y z A B C D E F G H I J K L M N O P Q R S T U V W X Y Z`
- Cifre (digit):
  `0 1 2 3 4 5 6 7 8 9`
- Caratteri speciali:
  ```
  ! ^ \& \* ( ) - + = { } | ~
  [ ] \ ; ' : " < > ? , . /
  ```

Le spaziature possono essere costituite da

- spazi: " "
- tabulazioni: "	"
- caratteri di nuova riga: "
"
- commenti:
  - racchiusi fra "/*" e "*/" (come in C):
    ```
    /* Questo commento e'
       su piu' righe      */
    ```
  - tra "//" e fine riga:
    ```
    // Questo commento e' su una sola riga
    ```

- Le entità di un programma C++ devono avere dei nomi che ne consentano l'individuazione
- Nel caso più semplice i nomi sono degli identificatori liberamente scelti
- Un identificatore è una parola iniziante con una lettera

Nota

- Il carattere "_" è una lettera
- Il C++ distingue tra maiuscole e minuscole (si dice che è case sensitive).
  - Es: `Fact` è diverso da `fact`

Nome	Abbreviazione	Sequenza di Escape
nuova riga	NL(LF)	\n
tabulazione orizzontale	HT	\t
tabulazione verticale	VT	\v
spazio indietro	BS	\b
ritorno carrello	CR	\r
avanzamento modulo	FF	\f
segnale acustico	BEL	\a
barra invertita	\	\\
apice	'	\'
virgolette	"	\"

- Un numero intero viene rappresentato da una sequenza di cifre, che vengono interpretate
  - in base decimale (default)
  - in base ottale se inizia con uno zero
  - in base esadecimale se inizia con '0x' (o '0X')
- Un numero reale (in virgola mobile) viene rappresentato facendo uso del punto decimale e della lettera 'e' (o 'E') per separare la parte in virgola fissa dall'esponente;
  - ad esempio: -1235.6e-2 rappresenta -12,356

Nota:

In un numero reale, la virgola viene rappresentata secondo la notazione anglosassone con un punto ".".

Principali operatori del C++

```
+    -    *    /    %    ^    &    |    ~
!    =    <    >    +=   -=   *=   /=   %=
^=   &=   |=   <<   >>   >>=  <<=  ==   !=
<=   >=   &&   ||   ++   --   ,    ->*  ->
.*   ::   ()   []   ?:
```

I/O standard

- Un programma comunica con l'esterno tramite uno o più flussi di caratteri (stream)
- Uno stream è una struttura logica costituita da una sequenza di caratteri, in numero teoricamente infinito, terminante con un apposito carattere che ne identifica la fine.
- Gli stream vengono associati (con opportuni comandi) ai dispositivi fisici collegati al computer (tastiera, video) o a file residenti sulla memoria di massa
- In C++ esistono i seguenti stream predefiniti
  - cin (stream standard di ingresso)
  - cout (stream standard di uscita)
  - cerr (stream standard di errore)
- Le funzioni che operano su questi stream sono in una libreria di ingresso/uscita e per usarle occorre la direttiva:
  `#include <iostream>`

endl="\n"

- `cin.eof()`: ritorna un valore diverso da 0 se lo stream cin ha raggiunto la sua fine (End Of File)
  - va usato sempre dopo almeno un'operazione di lettura
  - richiede un separatore dopo l'ultimo elemento letto
- `cin.fail()`: ritorna un valore diverso da 0 se lo stream cin ha rilevato uno stato di errore (e.g. stringa per `int`) o un end-of-file
  - non necessariamente usato dopo almeno un'operazione di lettura
  - non richiede un separatore dopo l'ultimo elemento letto
- `cin.clear()`: ripristina lo stato normale dallo stato di errore

Variabili e costanti

Le variabili sono caratterizzate da una quadrupla:

- il nome (identificatore)
- il tipo
- la locazione di memoria (l-value o indirizzo)
- il valore (r-value)

- Definizione:
  - Formato: `tipo identificatore;`
  - Esempio: `int x;`
  - Se (e solo se!) una variabile è definita esternamente ad ogni funzione, `main()` inclusa, (variabile globale) viene automaticamente inizializzata al valore 0
- Definizione con inizializzazione:
  - Formato: `tipo identificatore=exp;`
  - Esempio: `int x=3*2;`
  - Esempio: `int y=3*z; // z definita precedentemente`
- Dichiarazione:
  - Formato: `extern tipo identificatore;`
  - Esempio: `extern int x;`

- Sintassi:
  - Formato: `const tipo identificatore = exp;`
  - `exp` deve essere una espressione il cui valore deve poter essere calcolato in fase di compilazione
    (su alcuni compilatori è possibile inizializzare costanti a valore non costanti, ma il risultato è imprevedibile e varia a seconda dei casi $\Longrightarrow$ da evitare assolutamente)
- Esempi

```
const int kilo = 1024;
const double pi = 3.14159;
const int mille = kilo - 24;
```

- Serie di divisioni per 2 e analisi del resto
- Convertire 728_{10} in binario

728	0
364	0
182	0
91	1
45	1
22	0
11	1
5	1
2	0
1	1

Risultato: 1011011000_2

Prova: $2^9 + 2^7 + 2^6 + 2^4 + 2^3$
$= 512 + 128 + 64 + 16 + 8 = 728$

Esempi: operazioni tra interi positivi

```
 95 +     01011111       64+16+8+4+2+1          0 + 0 = 0
 54       00110110       32+16+4+2              1 + 0 = 1
-------------------------------------          1 + 1 = 0 (riporto 1)
149       10010101       128+16+4+1             0 + 1 = 1

149 -     10010101       128+16+4+1             0 - 0 = 0
095       01011111       64+16+8+4+2+1          1 - 0 = 1
-------------------------------------          1 - 1 = 0
 54       00110110       32+16+4+2              0 - 1 = 1 (prestato 1
                                                          col. sin.)
```

N.B.: Se eccede il range, il bit di riporto si perde (overflow) $\Longrightarrow$ aritmetica modulo 2^N

```
223 +     11011111       128+64+16+8+4+2+1
 54       00110110       32+16+4+2
-----------------------------------------
 21       00010101       16+4+1   (223+54-256=21)
```

Codifica Segno-valore

- Il primo bit rappresenta il segno, gli altri il valore
  - Esempio (8 bit): 10001001 rappresenta -9
- Comporta una doppia rappresentazione dello zero: 00000000 e 10000000
- I numeri rappresentati appartengono all'intervallo $[-2^{n-1}+1, 2^{n-1}-1]$ (ad esempio $[-2147483647, 2147483647]$ con 32 bit)
- Poco usato in pratica

Nota:

Occorre usare due diversi algoritmi per la somma a seconda che il segno degli addendi sia concorde o discorde

Codifica Complemento a 2

- Di gran lunga la più usata
- Un numero negativo è ottenuto calcolando il suo complemento (si invertono zeri e uni) e poi aggiungendo 1
- I numeri rappresentati appartengono all'intervallo $[-2^{n-1}, 2^{n-1} - 1]$ (ad es. $[-2147483648, 2147483647]$ con 32 bit)
- Rappresentazione ciclica: $-X$ in complemento a 2 si scrive come si scriverebbe $2^n - X$ nella rappresentazione senza segno
- Comporta un'unica rappresentazione dello zero: 00000000

Esempio:

- 11110111 in complemento a 2 rappresenta -9:

```
    9       compl.      +1
00001001 11110110 11110111
```

- 11110111 in rappresentazione senza segno rappresenta 247, cioè $2^8 - 9$

Interi senza segno vs. complemento a 2

Codifica	senza segno		complemento a 2	
00000000		0		0
00000001		1		1
⋮		⋮		⋮
00001001	$8+1=$	9	$8+1=$	9
⋮		⋮		⋮
01111111	$64+\ldots+2+1=$	127	$64+\ldots+2+1=$	127
10000000		128	$128-256=$	−128
10000001	$128+1=$	129	$128+1-256=$	−127
⋮		⋮		⋮
10001001	$128+8+1=$	137	$128+8+1-256=$	−119
⋮		⋮		...
11111111	$128+\ldots+1=$	255	$128+\ldots+1-256=$	−1

Esempi: Operazioni in Complemento a 2

```
  -9 +    11110111
   9      00001001
=  0      00000000

  -9 +    11110111
   8      00001000
= -1      11111111    => 11111110 => 00000001

  -9 +    11110111
  10      00001010
=  1      00000001

  -9 +    11110111
  -5      11111011
= -14     11110010   => 11110001 => 00001110
```

- Anche i numeri binari si possono esprimere con la virgola!
- Convertire 3.5_{10} in binario con virgola
  - Parte intera: $3_{10} \rightarrow 011_2$
  - Parte frazionaria: $0.5_{10} \rightarrow 2^{-1} \rightarrow 0.1_2$
  - Risultato: 11.1_2
- Prova inversa:
  - $11_2 \rightarrow 2^1 + 2^0 \rightarrow 2 + 1 \rightarrow 3_{10}$
  - $0.1_2 \rightarrow 2^{-1} \rightarrow 0.5_{10}$
  - Risultato: $3 + 0.5 = 3.5_{10}$

Per convertire la parte frazionaria bisogna moltiplicare per 2 e ogni volta prendere il numero prima della virgole e continuare a farlo:

Il procedimento consiste in questo:

– si prende il **numero senza parte intera** e lo si **moltiplica per due**, la **parte intera del risultato sarà la prima cifra dopo la virgola**

```
0,625 * 2 = 1,250 <- 1 sarà la prima cifra dopo la virgola
```

– **si ripete il procedimento con il risultato della moltiplicazione** del passaggio precedente.

```
0,25 * 2 = 0,5 <- 0 sarà la seconda cifra dopo la virgola
```

– bisogna **fermarsi** quando: o il **risultato della moltiplicazione è 1 o 0** (in quel caso si è scritto esattamente il numero da rappresentare) **o** ci si può fermare **al numero di cifre dopo la virgola voluto,** quando si lavora al computer di norma c'è un numero massimo di cifre memorizzabili (in questo caso il numero rappresentato non sarà esattamente il numero da rappresentare ma una sua approssimazione).

- ➔ Codifica numeri naturali positivi
  - ◆ Codifica binaria normale
- ➔ Codifica numeri naturali positivi e negativi
  - ◆ Codifica segno-valore
    - Il primo bit rappresenta il segno: 0=+; 1=-
    - Comporta doppia rappresentazione dello zero
    - Bisogna usare due algoritmi diversi per operazioni di somma e differenza in base che il segno dei numeri sia concorde o discorde
    - Poco usato in pratica
  - ◆ Codifica complemento a 2
    - Range diverso

        - Si trasforma numero in binario normalmente, si esegue il complemento (sostituendo 0 con 1 e viceversa), si somma 1

➔ Codifica numeri reali (con virgola)

- Si codifica prima la parte intera e poi la parte frazionaria
- Conversione IEEE754
    - Segno, esponente, mantissa

Tipi (interi, booleani, reali, caratteri)

Utilizzano stesso spazio di memoria e stessa codifica, sono soggetti alle stesse operazioni. Efficiente allocazione della memoria.

- Tipi fondamentali
    - Interi: short, int, long, long long (in ordine crescente di memoria)
        - Unsigned int (int senza segno)
    - Booleani: bool
    - Enumerati: enum
        - enum Giorno {LUN,MAR,MER,GIO,VEN,SAB,DOM}
          Giorno oggi = LUN
    - Carattere: char
        - Il “codice” (valore numerico associato) di ogni carattere dipende dal tipo di codifica (ASCII, etc)
    - Reali: float, double, long double
- Tipi derivati
    - Array, puntatori… etc.

Nel C++ i tipi sono distinti in:

- i tipi fondamentali
    - che servono a rappresentare informazioni semplici
    - Esempio: i numeri interi o i caratteri (int, char, ...)
- i tipi derivati
    - permettono di costruire strutture dati complesse
    - si costruiscono a partire dai tipi fondamentali, mediante opportuni costruttori (array, puntatori, ...)

I Tipi Fondamentali in C++

- i tipi interi: `int`, `short`, `long`, `long long`
- i tipi Booleani: `bool`
- i tipi enumerativi: `enum`
- il tipo carattere: `char`
- i tipi reali: `float`, `double`, `long double`

I primi quattro sono detti tipi discreti (hanno un dominio finito)

I tipi interi (con segno)

- I tipi interi (con segno) sono costituiti da numeri interi compresi tra due valori estremi, a seconda dell'implementazione
  - tipicamente codificati in complemento a 2 con N bit (N = 16, 32, ...)
  - appartengono all'intervallo $[-2^{N-1}, 2^{N-1} - 1]$
- Quattro tipi, in ordine crescente di dimensione
  `short [int]`, `int`, `long [int]`, `long long [int]`
- Dimensioni dipendenti dall'implementazione (macchina, s.o., ...)
  - sizeof(short) $\leq$ sizeof(int) $\leq$ sizeof(long) $\leq$ sizeof(long long)
    (sizeof() restituisce la dimensione del tipo in byte)
  - `short` tipicamente non ha più di 16 bit: $[-32768, 32767]$
  - `long` tipicamente ha almeno 32 bit: $[-2147483648, 2147483647]$

Il valore di un'espressione intera non esce dal range $[-2^{N-1}, 2^{N-1} - 1]$ perché i valori "ciclano": il successore di 2147483647 è -2147483648

I tipi interi (senza segno)

- Il tipo `unsigned ...` rappresenta numeri interi non negativi di varie dimensioni
  - codifica interi senza segno a N bit (N=16,32, ...), range $[0, 2^N - 1]$
  - tipicamente usati molto poco, come rappresentazioni di sequenze di bit (applicazioni in elettronica)

```
unsigned int x=1232;
unsigned short int x=567;
unsigned long int x=878678687;
```

Il valore di un'espressione unsigned non esce mai dal range $[0, 2^N - 1]$ perché, quando il valore aritmetico X esce da tale range, il valore restituito è X modulo 2^N.

Siano x e y rappresentati su 16 bit

```
x:    0000000000001100   (12)
y:    0000000000001010   (10)
x|y:  0000000000001110   (14)
x&y:  0000000000001000   ( 8)
x^y:  0000000000000110   ( 6)
~x:   1111111111110011   (65523 oppure -13 )
x>>2: 0000000000000011   ( 3)
x<<2: 0000000000110000   (48)
```

Istruzioni elementari

Forma compatta	Forma estesa
`x += y`	`x = x+y`
`x -= y`	`x = x-y`
`x *= y`	`x = x*y`
`x /= y`	`x = x/y`
`x %= y`	`x = x%y`

L'uso della forma compatta è tipicamente più efficiente
(su alcune architetture consente di utilizzare in modo ottimale funzionalità della CPU)

- `x++`:
  - incrementa `x` di un'unità
  - denota il valore di `x` prima dell'incremento
- `x--`:
  - decrementa `x` di un'unità
  - denota il valore di `x` prima del decremento
- `++x`:
  - incrementa `x` di un'unità
  - denota il valore di `x` dopo l'incremento
- `--x`:
  - decrementa `x` di un'unità
  - denota il valore di `x` dopo il decremento

Nota

L'uso della forma compatta: "`x++;`" è tipicamente più efficiente della forma estesa corrispondente: "`x = x + 1;`"

Il tipo Booleano

Il C++ prevede un tipo Booleano `bool`:

- il valore falso è rappresentato dalla costante `false` (equivalente a 0)
- il valore vero è rappresentato dalla costante `true` (equivalente ad un valore intero diverso da 0)
- si può usare a tal scopo anche il tipo `int`
- operatori Booleani: `!` (not), `&&` (and), `||` (or)

x	y	!x	&&	||
false	false	true	false	false
false	true	true	false	true
true	false	false	false	true
true	true	false	true	true

- `!!x` $\Longleftrightarrow$ `x`,
  `!(x || y)` $\Longleftrightarrow$ `(!x && !y)`,
  `!(x && y)` $\Longleftrightarrow$ `(!x || !y)`,

- L'operatore `sizeof`, può essere prefisso a:
  - una variabile (esempio: `sizeof (x)`)
  - una costante (esempio: `sizeof ('a')`)
  - al nome di un tipo (esempio: `sizeof (double)`)
- Può essere usato in alcuni casi con o senza parentesi
  - `sizeof x` Ok
  - `sizeof double` Ko
- Restituisce un intero rappresentante la dimensione in byte
- È applicabile a espressioni di qualsiasi tipo (non solo ai tipi fondamentali)

- Il programmatore può richiedere una conversione esplicita di un valore da un tipo ad un altro (casting)
- Esistono due notazioni:
  - prefissa. Esempio:
    `int i = (int) 3.14;`
  - funzionale. Esempio:
    `double f = double(3)`

- Promozione: conversione da un tipo ad uno simile più grande
  - `short` $\Longrightarrow$ `int` $\Longrightarrow$ `long` $\Longrightarrow$ `long long`,
    `float` $\Longrightarrow$ `double` $\Longrightarrow$ `long double`
  - garantisce di mantenere lo stesso valore
- Conversioni tra tipi compatibili
  - Conversione da tipo reale a tipo intero: il valore viene troncato
    `int x = (int) 4.7` $\Longrightarrow$ `x==4`,
    `int x = (int) -4.7` $\Longrightarrow$ `x==-4`
  - Conversione da tipo intero a reale: il valore può perdere precisione
    `float y = float(2147483600); // 2^31-48`
    $\Longrightarrow$ `y == 2147483648.0; // 2^31`

Istruzioni strutturate (sequenze, condizioni, cicli)

Espressione condizionale

- Sintassi: `exp1 ? exp2 : exp3` :
  Se exp1 è vera equivale a exp2, altrimenti equivale a exp3

Esempio

```
prezzo = valore * peso * (peso>10) ? 0.9 : 1;
```

- se il peso è maggiore di 10, equivale a:
  `prezzo = valore * peso * 0.9;`
- altrimenti, equivale a
  `prezzo = valore * peso * 1;`

Gli Invarianti di un Ciclo (Loop Invariant)

- Tecnica per la verifica di correttezza dei cicli (proprietà P)
- Idea: suddividere la proprietà desiderata P della correttezza del ciclo in una sequenza di affermazioni $P_0, P_1, \ldots P_n$, in modo che:
  (1) P_0 sia vera immediatamente prima che il ciclo inizi
  (dopo l'inizializzazione!)
  (2) per ogni indice di ciclo $i \in \{1, \ldots, n\}$:
  se P_{i-1} è vera prima dell'inizio del ciclo i-esimo (ed è verificata la condizione di permanenza del ciclo), allora P_i è vera alla fine del ciclo i-esimo
  (e quindi immediatamente prima dell'inizio del ciclo $(i+1)$-esimo)
  (3) Alla fine dell'ultimo ciclo (n-esimo), P_n (e la negazione della condizione di permanenza) implica la proprietà P
- Tipicamente (2) è il passo più critico
- P_i a volte ovvie, a volte molto complesse (or, if-then-else, ...)
  $\Longrightarrow$ problema indecidibile in generale
- Talvolta necessarie variabili ausiliarie addizionali
- Talvolta si adottano convenzioni per gestire il caso $i = 0$:
  (la somma di 0 elementi è 0, il prodotto di 0 elementi è 1, ...)

Puntatori e Riferimenti, Algebra dei puntatori

- Dai tipi fondamentali, attraverso vari meccanismi, si possono derivare tipi più complessi
- I principali costrutti per costruire tipi derivati sono:
  - i riferimenti
  - i puntatori
  - gli array
  - le strutture
  - le unioni
  - le classi

Riferimento a

Sintassi: tipo & id = exp
Serve a dare più nomi ad una variabile
exp deve essere dotata di indirizzo
è obbligatorio inizializzare dopo la definizione.

Costante riferimento

Non permette di modificare ma solo di leggere dal riferimento.

Address of

Sintassi &
Ritorna l'indirizzo dell'espressione che precede

Puntatori

Sintassi: tipo *id = &x
Il puntatore contiene un indirizzo di memoria

Operatore di dereference

Sintassi: *puntatore
Ritorna il valore della variabile dell'area di memoria "puntata"

Puntatori a costanti

Sintassi: const tipo *id
Non permette di modificare l'oggetto puntato tramite dereference
Non rende l'oggetto puntato una costante, si possono creare puntatori a costanti che puntano ad variabili non costanti

Costanti puntatori

Sintassi: tipo *const id = exp
Non permette di cambiare l'oggetto puntato

Costanti puntatori a costante

Sintassi: const tipo *const id = exp
Non permette né di cambiare né di modificare l'oggetto puntato

Puntatori a `void` (cenni)

- In alcuni casi è utile avere una variabile puntatore che possa puntare ad entità di tipo diverso;
- Tale variabile viene dichiarata di tipo "puntatore a `void`" cioè a tipo non specificato

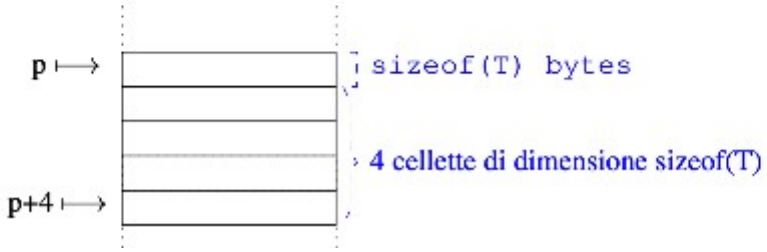
```
int i; int *pi=&i;
char c; char *pc=&c;
void *tp;
tp = pi;            // punta a int
*(int*)tp=3;
tp = pc;            // punta a char
*(char*)tp='C';
```

Aritmetica di Puntatori ed Indirizzi

Gli indirizzi e i puntatori hanno un'aritmetica:
se `p` è di tipo `T*` e `i` è un intero, allora:

- `p+i` è di tipo `T*` ed è l'indirizzo di un oggetto di tipo `T` che si trova in memoria dopo `i` posizioni di dimensione **`sizeof`**`(T)`
- analogo discorso vale per `p++`, `++p`, `p--`, `--p`, `p+=i`, ecc.

$\Longrightarrow$ `i` viene implicitamente moltiplicato per **`sizeof`**`(T)`



Funzioni e passaggi di parametri

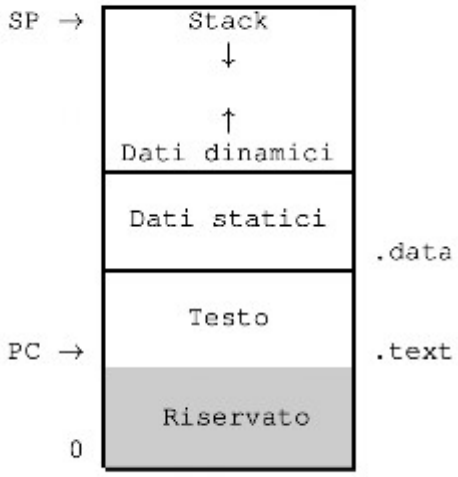
- Una funzione è un sotto-programma che può essere utilizzato ripetutamente in un programma, o in programmi diversi
- Una libreria è un insieme di funzioni precompilate.
- Alcune librerie C++ sono disponibili in tutte le implementazioni e con le stesse funzioni (ad es. `cmath`)
- Una libreria è formata da una coppia di file:
  - un file di intestazione (header) contenente le dichiarazioni dei sotto-programmi stessi
  - un file oggetto contenente le funzioni compilate
- Per utilizzare in un programma le funzioni in una libreria bisogna:
  - includere il file di intestazione della libreria con la direttiva **`#include`** `<nomelibreria>`
  - in alcuni casi, indicare al linker il file contenente le funzioni compilate della libreria
  - introdurre nel programma chiamate alle funzioni della libreria

- Libreria `<cmath>`: funzioni matematiche (da **`double`** a **`double`**)
  - `fabs(x)`: valore assoluto di tipo float
  - `sqrt(x)`: radice quadrata di x
  - `pow(x,y)`: eleva x alla potenza di y
  - `exp(x)`: eleva e alla potenza di x
  - `log(x)`: logaritmo naturale di x
  - `log10(x)`: logaritmo in base 10 di x
  - `sin(x)` e `asin(x)`: seno e arcoseno trigonometrico
  - `cos(x)` e `acos(x)`: coseno e arcocoseno trigonometrico
  - `tan(x)` e `atan(x)`: tangente e arcotangente trig.
  - ...
- possono essere usate con tutti gli altri tipi numerici tramite conversione implicita o esplicita

- Libreria `<cctype>`, funzioni di riconoscimento (da **`char`** a **`bool`**):
  - `isalnum(c)`: carattere alfabetico o cifra decimale
  - `isalpha(c)`: carattere alfabetico
  - `iscntrl(c)`: carattere di controllo
  - `isdigit(c)`: cifra decimale
  - `isgraph(c)`: carattere grafico, diverso da spazio
  - `islower(c)`: lettera minuscola
  - `isprint(c)`: carattere stampabile, anche spazio
  - `isspace(c)`: spazio,salto pagina,nuova riga o tab.
  - `isupper(c)`: lettera maiuscola
  - `isxdigit(c)`: cifra esadecimale
  - ...
- Libreria `<cctype>`, funzioni di conversione (da **`char`** a **`char`**):
  - `tolower(c)`: se `c` è una lettera maiuscola restituisce la corrispondente lettera minuscola, altrimenti restituisce `c`
  - `toupper(c)`: come sopra ma in maiuscolo
  - ...

Area di memoria allocata ad un'esecuzione di un programma:

- Area programmi: destinata a contenere le istruzioni (in linguaggio macchina) del programma
- Area dati statici: destinata a contenere variabili globali o allocate staticamente e le costanti del programma.
- Area heap: destinata a contenere le variabili dinamiche (di dimensioni non prevedibili a tempo di compilazione) del programma.
- Area stack: destinata a contenere le variabili locali e i parametri formali delle funzioni del programma.



- I parametri formali e le variabili locali a una funzione sono memorizzate in un'area riservata della memoria, detta stack
- Modello di memoria concettualmente analogo a quello di una "pila" ("stack"):
  - quando una funzione viene chiamata, il blocco di memoria necessario per contenere i suoi parametri formali e variabili locali viene allocato "sopra" quello della funzione che la chiama
  - quando la funzione termina, tale blocco viene reso di nuovo disponibile
  - politica di gestione "last in first out" (LIFO)

In C++ esistono tre modalità passaggio di parametri a una funzione:

- per valore
- per riferimento
- per puntatore

(Spesso le ultime due sono confuse in letteratura, perché hanno finalità simili.)

- Definizione di parametri formali analoga a definizione di variabili
  - Sintassi lista dei parametri: `( tipo identificatore, ...)`
  - Es: `int fact(int n) {...}`
- Simile a definire una nuova variabile locale e assegnarle il valore dell'espressione del parametro attuale.
  - Es: `fact(3*x); //simile a: int n = 3*x;`
- Il parametro formale acquisisce il valore del parametro attuale
  - il parametro attuale può essere un'espressione senza indirizzo
  - può essere di tipo diverso compatibile $\Longrightarrow$ conversione implicita
- L'informazione viene (temporaneamente) duplicata
  $\Longrightarrow$ possibile spreco di tempo CPU e memoria
- Se modifico il parametro formale, il parametro attuale non viene modificato
  $\Longrightarrow$ passaggio di informazione solo dalla chiamante alla chiamata

Il passaggio di parametri per riferimento

- Definizione di parametri formali simile a definizione di riferimenti
  - Sintassi lista dei parametri: `( tipo & identificatore,...)`
  - Es: `int swap(int & n, int & m) {...}`
- Simile a definire un riferimento "locale" ad un'espressione dotata di indirizzo
  - Es: `swap(x,y); //simile a: int & n=x; int & m=y`
- Il parametro è un riferimento al parametro attuale
  - il parametro attuale deve essere un'espressione dotata di indirizzo
  - deve essere dello stesso tipo
- L'informazione non viene duplicata
  $\Longrightarrow$ evito possibile spreco di tempo CPU e memoria
- Se modifico il parametro formale, modifico il parametro attuale
  $\Longrightarrow$ passaggio di informazione anche dalla chiamata alla chiamante

Passaggio di parametri per riferimento costante

- È possibile definire passaggi per riferimento in sola lettura (passaggio per riferimento costante)
  - Sintassi: `(const tipo & identificatore, ...)`
  - Es: `int fact(const int & n,...) {...}`
- Riferimento: l'informazione non viene duplicata
  $\Longrightarrow$ evito possibile spreco di tempo CPU e memoria
- Non permette di modificare `n`!
  - Es: `n = 5; //ERRORE!`
  - $\Longrightarrow$ passaggio di informazione solo dalla chiamante alla chiamata
  - $\Longrightarrow$ solo un input alla funzione
- Usato per passare in input alla funzione oggetti "grossi"
  - efficiente (no spreco di CPU e memoria)
  - evita errori
  - permette di individuare facilmente gli input della funzione

Il passaggio di parametri per puntatore

- Definizione di parametri formali: puntatori passati per valore
  - Sintassi lista dei parametri: `( tipo * identificatore,...)`
  - Es: `int swap(int * pn, int * pm) {...}`
  - N.B.: nella chiamata, si passa l'indirizzo dell'oggetto passato
- Simile a definire un puntatore "locale" ad un'espressione dotata di indirizzo
  - Es: `swap(&x,&y);//simile a: int *pn=&x; int *pm=&y`
- Il parametro è un puntatore al(l'oggetto il cui indirizzo è dato dal) parametro attuale
  - che deve essere un'espressione dotata di indirizzo
  - che deve essere dello stesso tipo
- L'informazione non viene duplicata
  $\Longrightarrow$ evito possibile spreco di tempo CPU e memoria
- Se modifico il parametro formale, modifico il parametro attuale
  $\Longrightarrow$ passaggio di informazione anche dalla chiamata alla chiamante

$\Longrightarrow$ effetto simile al passaggio per riferimento (vedi C)

Funzioni che restituiscono un riferimento

- Restituisce un riferimento ad un'espressione (con indirizzo)
  - l'espressione deve riferirsi ad un oggetto del chiamante (es. un parametro formale passato per riferimento, un elemento di un array)
  - deve essere dello stesso tipo
- La chiamata è un'espressione dotata di indirizzo!

```
int& max(int& x,int& y) //restituisce un riferimento
{return (x > y ? x : y);}//x,y riferimenti a oggetti
                         //non locali
(...)
int m=44, n=22;
max(m,n) = 55;     //cambia il valore di m da 44 a 55
```

Funzioni con argomenti di default (cenni)

- In C++ è possibile fornire parametri opzionali, con valori di default
  - permette chiamate con liste di parametri attuali ridotte
  - i parametri opzionali devono essere gli ultimi della lista
  - il match viene effettuato da sinistra a destra

```
double p(double, double, double =0, double =0, double =0);

  cout << p(x, 7) << endl;
  cout << p(x, 7, 6) << endl;
  cout << p(x, 7, 6, 5) << endl;
  cout << p(x, 7, 6, 5, 4) << endl;

double p(double x, double a0, double a1, double a2, double a3)
{ return a0 + (a1 + (a2 + a3*x)*x)*x; }
```

Per valore

Viene copiato il valore del parametro attuale nel parametro locale e temporaneo.
Il parametro attuale può essere un'espressione senza indirizzo.
L'informazione viene temporaneamente duplicata.
Modificando il parametro formale non altero il parametro attuale.

Per riferimento

Sintassi lista dei parametri (tipo & id).
Il parametro attuale DEVE essere un'espressione dotata di indirizzo.
L'informazione non viene duplicata.
L'espressione deve essere dello stesso tipo.
Nel caso si voglia fare una funzione che abbia più di un return, si fa direttamente una funzione void in cui passano due variabili per riferimento che conterranno il return.

Per riferimento costante

Sintassi (const tipo & identificatore).
Niente duplicazione
Non permette di modificare, può essere usata solo come input

Per puntatori

Sintassi (tipo *id)
Passa i puntatori
Niente duplicazioni
Nella chiamata devo passare il "address of"
Vantaggi in termini di CPU
Rischio di confusione del codice
Aliasing (tanti nomi per la stessa area di memoria)
Solo espressioni dotate di indirizzo
C permette solo passaggio per puntatore (non per riferimento)

Funzioni con argomenti di default (cenni)

- In C++ è possibile fornire parametri opzionali, con valori di default
  - permette chiamate con liste di parametri attuali ridotte
  - i parametri opzionali devono essere gli ultimi della lista
  - il match viene effettuato da sinistra a destra

```
double p(double, double, double =0, double =0, double =0);

  cout << p(x, 7) << endl;
  cout << p(x, 7, 6) << endl;
  cout << p(x, 7, 6, 5) << endl;
  cout << p(x, 7, 6, 5, 4) << endl;

double p(double x, double a0, double a1, double a2, double a3)
{ return a0 + (a1 + (a2 + a3*x)*x)*x; }
```

Funzioni ricorsive

Funzioni ricorsive

- In C++ una funzione può invocare se stessa (funzione ricorsiva)
- ... o due o più funzioni possono chiamarsi a vicenda (funzioni mutualmente ricorsive)
- Formulare alcuni problemi in maniera ricorsiva risulta naturale:
  - il fattoriale: $0! \stackrel{def}{=} 1$; $n! \stackrel{def}{=} n \cdot (n-1)!$
  - pari/dispari: $even(n) \iff odd(n-1)$; $odd(n) \iff even(n-1)$;
  - espressioni: $somma \stackrel{def}{=} numero$; $somma \stackrel{def}{=} (somma + somma)$
- Due componenti:
  - una o più condizioni di terminazione
  - una o più chiamate ricorsive
- Intrinseco rischio di produrre sequenze infinite
  - Analoghe considerazioni rispetto ai cicli
- Alcune "insidie" computazionali
  - Es: funzione di Fibonacci: $f_0 \stackrel{def}{=} 1$; $f_1 \stackrel{def}{=} 1$; $f_n \stackrel{def}{=} f_{n-1} + f_{n-2}$

Ricorsione fortemente collegata al principio di induzione matematico.

Da ricorsione in coda a iterazione (caso void)

```
void F(int x,...) {
 if (CasoBase(x,...))
  {IstrBase(...);}
 else {
  Istr(...);
  x=agg(x,...);
  F(x,...);
}}
```

$\iff$

```
void F(int x,...) {
 while (!CasoBase(x,...)){
  Istr(...);
  x=agg(x,...);
 }
 {IstrBase(...);}
}
```

Da ricorsione in coda a iterazione (caso generale)

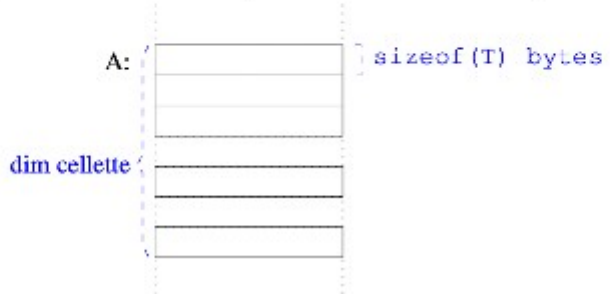
```
type F(int x,...) {
 if (CasoBase(x,...))
  res = IstrBase(...);
 else {
  Istr(...);
  x=agg(x,...);
  res = F(x,...);
 }
 return res;
}
```

$\iff$

```
type F(int x,...) {
 while (!CasoBase(x,...)){
  Istr(...);
  x=agg(x,...);
 }
 res = IstrBase(...);
 return res;
}
```

Array, array ordinati, array multidimensionali

- Array: sequenza finita di elementi consecutivi dello stesso tipo.
- Il numero di elementi di un array (dimensione) è fissata a priori
- Per un array di tipo `T` e dimensione `dim`, il compilatore alloca `dim` cellette consecutive di **`sizeof`**`(T)` bytes (allocazione statica)
- Un array rappresenta l'indirizzo del primo elemento della sequenza



- Sintassi definizione:
  - `tipo id[dim];`
  - `tipo id[dim]={lista_valori};`
  - `tipo id[]={lista_valori};`
- Esempi:

```
double a[25]; //array di 25 double
const int c=2;
char b[2*c]={'a','e','i','o'};  // dimensione 4
char d[]={'a','e','i','o','u'}; // dimensione 5
```

- La dimensione dell'array deve essere valutabile al momento della compilazione:
  - esplicitamente, tramite l'espressione costante `dim`
  - implicitamente, tramite la dimensione della lista di inizializzazione
- Se mancano elementi nella lista di inizializzazione, il corrispondente valore viene inizializzato allo zero del tipo T

- `g++ -fsanitize=bounds -fsanitize=bounds-strict` può aiutare ad identificare a run-time la maggior parte dei casi di errore di array subscripting out of bounds!

Funzione con parametri di tipo array

- Una funzione può avere un parametro formale del tipo "array di T"
  - Es: `float sum(float v[],int n) {...}`
  - tipicamente si omette la dimensione ("`float v[]`", non "`float v[dim]`")
  - tipicamente associato al numero di elementi effettivamente utilizzati (dimensione virtuale)
- Il corrispondente parametro attuale è un array di oggetti di tipo `T`
  - Es:

```
float a[DIM]; int n;
(...)
x = sum(a,n);
```

- Nel passaggio, gli elementi dell'array non vengono copiati
  - N.B. viene copiato solo l'indirizzo del primo elemento
  - Equivalente a passare gli elementi dell'array per riferimento
  - È possibile impedire di modificarli usando la parola chiave **`const`**

Nota

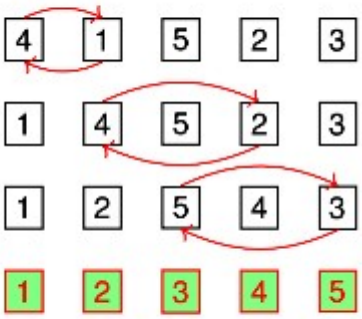
Con allocazione statica di array, tipicamente si definiscono array sufficientemente grandi, e poi se ne usa di volta in volta solo una parte.

Passaggio di parametri array costanti

- È possibile definire passaggi di array in sola lettura (passaggio di array costante)
  - Sintassi: `(const tipo identificatore [], ...)`
  - Es: `int print(const int v[],...) (...) {...}`
- Passaggio di array: il contenuto dell'array non viene duplicato
  $\Longrightarrow$ evito possibile spreco di tempo CPU e memoria
- Non permette di modificare gli elementi di `v`!
  - Es: `v[3] = 5; //ERRORE!`
  - $\Longrightarrow$ passaggio di informazione solo dalla chiamante alla chiamata
  - $\Longrightarrow$ solo un input alla funzione
- Usato per passare array in input alla funzione
  - efficiente (no spreco di CPU e memoria)
  - evita errori
  - permette di individuare facilmente gli input della funzione

Ordinamento per selezione: Selection Sort

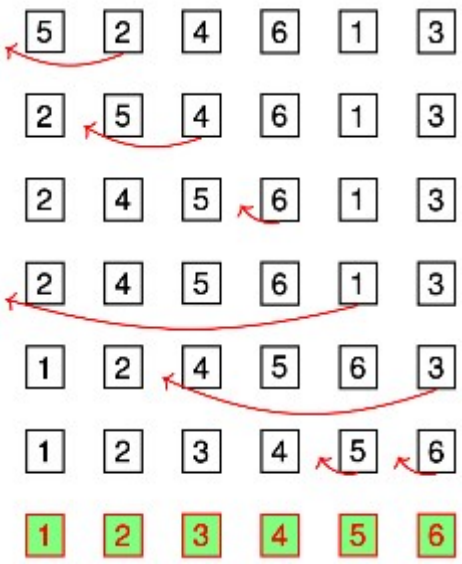
- Cerco elemento più piccolo dell'array e lo scambio con il primo elemento dell'array.
- Cerco il secondo elemento più piccolo dell'array e lo scambio con il secondo elemento dell'array.
- Proseguo in questo modo fintanto che l'array non è ordinato.



Selection sort:
{ `SORT/selection_sort.cc` }

Ordinamento per inserzione: Insertion Sort

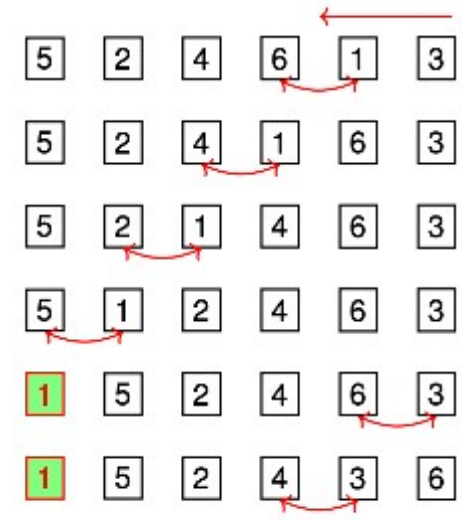
- È il metodo usato dai giocatori di carte per ordinare in mano le carte.
- Considero un elemento per volta e lo inserisco al proprio posto tra quelli già considerati (mantenendo questi ultimi ordinati).
  - L'elemento considerato viene inserito nel posto rimasto vacante in seguito allo spostamento di un posto a destra degli elementi più grandi.



Insertion sort:
{ SORT/insertion_sort.cc }

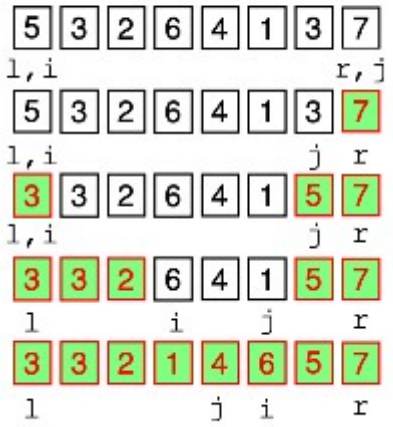
Ordinamento a bolle: Bubble Sort

- Si basa su scambi di elementi adiacenti se necessari, fino a quando non è più richiesto alcuno scambio e l'array risulta ordinato.



bubble sort:
{ SORT/bubblesort_nocomment.cc }

Ordinamento QuickSort (III)



- Partizioniamo a partire da `A[0] = 5`.
- Gli elementi dell'array precedenti ad `A[i]` sono minori od uguali a `5`.
- Gli elementi dopo `A[j]` sono maggiori od uguali a `5`.
- Ripartizioniamo i sotto array da `l` a `i-1`, e da `i+1` a `r`.

Algoritmo di ordinamento: Shell Sort

- La lentezza dell'algoritmo Insertion Sort risiede nel fatto che le operazioni di scambio avvengono solo tra elementi contigui.
  - Esempio: se l'elemento più piccolo è in fondo all'array, occorrono N scambi per posizionarlo al posto giusto.
- Per migliorare questo algoritmo è stato pensato l'algoritmo Shell Sort.
- L'idea è quella di organizzare l'array in modo che esso soddisfi la proprietà per cui gli elementi aventi tra loro distanza `h` costituiscono una sequenza ordinata, indipendentemente dall'elemento di partenza.
  - Se si applica l'algoritmo con una sequenza di `h` che termina con `1`, si ottiene un file ordinato.

```
void ShellSort(int A[],
               int l, int r)
  int h;
  for(h = 1; h <= (r-l)/9;
             h = 3*h+1);
  for( ; h > 0; h /= 3)
   for(int i = l+h;
         i <= r; i++) {
     int j = i;
     int v = A[i];
     while((j >= l + h) &&
           (v < A[j-h])) {
       A[j] = a[j-h];
       j = j - h;
     }
     A[j] = v;
   }
}
```

Array multidimensionali

- In C++ è possibile dichiarare array i cui elementi siano a loro volta degli array, generando degli array multidimensionali (matrici)
- Sintassi:

```
tipo id[dim1][dim2]...[dimN];
tipo id[dim1][dim2]...[dimN]={lista_valori};
tipo id[][dim2]...[dimN]={lista_valori};
```

$\Longrightarrow$ un array multidimensionale $dim_1 \cdot \ldots \cdot dim_n$ può essere pensato come un array di dim_1 array multidimensionali $dim_2 \cdot \ldots \cdot dim_n$

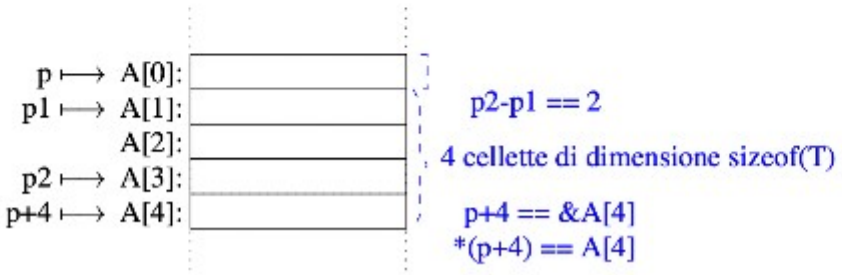
- Esempio:

```
int MAT[2][3];
int MAT[2][3] = {{1, 2, 3}, {4, 5, 6}};
int MAT[][3]  = {{1, 2, 3}, {4, 5, 6}};
```

- Le dimensioni devono essere valutabili durante la compilazione
- Elementi mancanti in inizializzazione (se presente) vengono sostituiti da zeri

Puntatori ad elementi di un array

- Se un puntatore `p` punta al primo elemento di un array `A`, l'espressione `p+i` è l'indirizzo dell'i-esimo elemento dell'array `A`
  $\Longrightarrow$ valgono `p+i == &(A[i])` e `*(p+i) == A[i]`
- Se due puntatori dello stesso tipo `p1, p2` puntano ad elementi di uno stesso array, `p2-p1` denota il numero di elementi compresi tra `p1` e `p2` (`p2-p1` negativo se `p2` precede `p1`)



Passaggio di Array tramite Puntatore

- Nelle chiamate di funzioni viene passato solo l'indirizzo alla prima locazione dell'array,

$\Longrightarrow$ un parametro formale array può essere specificato usando indifferentemente la notazione degli array o dei puntatori

- Le seguenti scritture sono equivalenti (array semplici):

```
int f(int arr[dim]);
int f(int arr[]);
int f(const int *arr);
```

- Le seguenti scritture sono equivalenti (array multi-dimensionali):

```
int f(int arr[dim1][dim2]...[dimN]);
int f(int arr[][dim2]...[dimN]);
int f(const int *arr[dim2]...[dimN]);
```

Problema importante

Una funzione può restituire un array (allocato staticamente)?

- Sì, ma solo se è allocato staticamente esternamente alla funzione! (es. parametro formale, array globale)
- No, se è allocato staticamente internamente alla funzione!

Stringhe e trattamento di testi

- Una stringa C è un array di **char**, il cui ultimo elemento è il carattere nullo (`'\0'`)
  - Esempio: `char stringa[]= "Ciao";`
  - Equivalente a `char stringa[]= {'C','i','a','o','\0'};`

  $\Longrightarrow$ contiene 5 elementi: i 4 caratteri della costante stringa e il carattere nullo che viene inserito automaticamente

$\Longrightarrow$ La dimensione dell'array deve essere maggiore di almeno 1 rispetto al numero di caratteri che si vuole rappresentare

Nota:

dato che negli array non è definito l'assegnamento, l'uso di una costante stringa per specificare il valore di una stringa è permesso solo nell'inizializzazione

Input e Output di Stringhe

- Gli operatori di I/O >>, << operano direttamente su stringhe!
- L' operatore di ingresso >>:
  1. legge caratteri da `cin`
  2. li memorizza in sequenza finché non incontra una spaziatura (che non viene letta)
  3. memorizza `'\0'` nella stringa dopo l'ultimo carattere letto
  4. termina l'operazione
- L' operatore di uscita << scrive in sequenza su `cout` i caratteri della stringa, fino al primo `'\0'` (che non viene scritto)

Esempio

Legge una stringa di al più 255 caratteri e la stampa:

```
char buffer[256];
cin >> buffer;
cout << buffer;
```

Alcune funzioni utili della libreria <iostream> II

Nelle funzioni seguenti `s` è una stringa, `c` è un carattere e `n` un intero:

- `cin.getline(s,n)`: legge da `cin` una riga in `s` fino a capo linea, per un massimo di `n-1` caratteri (lo `'\n'` non viene letto)
  - restituisce (un oggetto equivalente a) `0` se incontra `eof`
- `cin.get(c)`: legge da `cin` in `c` un singolo carattere (spaziature comprese), restituisce `c` (`'\0'` se `c` è `eof`)
- `cout.put(c)`: scrive su `cout` il singolo carattere `c`

Alcune funzioni utili della libreria <cstring>

Nelle funzioni che seguono `s` e `t` sono stringhe e `c` è un carattere:

- `strlen(s)`: restituisce la lunghezza di `s` escluso `'\0'`
- `strchr(s,c)`: restituisce un puntatore alla prima occorrenza di `c` in `s`, oppure **NULL** se `c` non si trova in `s`
- `strrchr(s,c)`: come sopra ma per l'ultima occorrenza di `c` in `s`
- `strstr(s,t)`: restituisce un puntatore alla prima occorrenza della sottostringa `t` in `s`, oppure **NULL** se `t` non si trova in `s`
- `strcpy(s,t)`: copia `t` in `s` e restituisce `s`
- `strncpy(s,t,n)`: copia `n` caratteri di `t` in `s` e restituisce `s`. Se non c'è lo `'\0'` negli `n` caratteri la stringa `s` non è ben formata!
- `strcat(s,t)`: concatena `t` al termine di `s` e restituisce `s`
- `strncat(s,t,n)`: concatena `n` caratteri di `t` al termine di `s` e restituisce `s`. La stringa di destinazione contiene sempre `'\0'`, vengono copiati `n` caratteri e sempre aggiunto `'0'`).
- `strcmp(s,t)`: restituisce un valore negativo, nullo o positivo se `s` è alfabeticamente minore, uguale o maggiore di `t`
- Prestare molta attenzione nelle funzioni sopra allo `'\0'` e che dimensioni delle stringhe di destinazioni siano in grado di contenere risultato!

I/O su files (argc & argv)

Argomenti da linea di comando

- In C++ è possibile passare ai programmi argomenti (es. valori numerici, nomi di file,...) direttamente da linea di comando

  `>./a.out 1000 22.5 miofile`

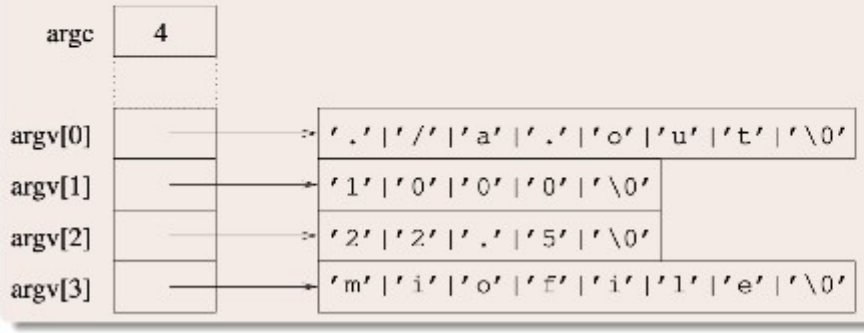
- Possibile tramite due parametri formali predefiniti della funzione `main`:
  `int main (int argc, char * argv[]) {}`
  - l'intero `argc`, in cui viene automaticamente copiato il numero delle parole della riga di comando ("`./a.out`" o analogo inclusa)
  - l'array di puntatori a caratteri (stringhe) `argv` in cui vengono automaticamente copiate le parole della linea di comando

Nota

Gli argomenti sono stringhe: se rappresentano numeri, devono essere convertiti tramite le funzioni `atoi` o `atof` della libreria `<cstdlib>`

Argomenti da linea di comando II

```
int main (int argc, char * argv[])
{...}
>./a.out 1000 22.5 miofile
```



Stream e I/O su File di Testo

- In C++ sono possibili operazioni di I/O direttamente da file di testo (senza usare <, >) tramite la libreria `<fstream>`
- È possibile definire stream, a cui associare (i nomi di) file di testo.
- Lo stream viene aperto e associato al nome di un file tramite il comando `open`, in tre possibili modalità
  - lettura da file,
  - scrittura su file,
  - scrittura a fine file (append).
- Lo stream può essere utilizzato per tutte le operazione di letture [resp. scrittura] a seconda della modalità di apertura
- Uno stream, quando è stato utilizzato, può essere chiuso mediante la funzione `close`

`fstream` "eredita" da `iostream` sostanzialmente tutti i suoi operatori e funzioni di lettura e scrittura, nelle rispettive modalità
(Es: `<<`, `>>`, `get`, `put`, `getline`, `eof`, `fail`, `clear`,...)

Operazioni su Stream: Sintassi

- Definizione di uno stream:
  - Sintassi: `fstream nomestream;`
  - Esempio: `fstream myin,myout,myapp;`
- Apertura di uno stream:
  - Sintassi: `nomestream.open(nomefile,modo);`
  - Es:
    ```
    myin.open("ingresso.txt",ios::in); //lettura
    myout.open("uscita.txt",ios::out); //scrittura
    myapp.open("uscita2.txt",ios::out|ios::app); //app.
    ```
- Utilizzo di uno stream:
  - Sintassi: analoga a `cin` e `cout`
  - Es:
    ```
    myin >> a; myout << x; myin.get(c); myapp.put(c);...
    ```
- Chiusura di uno stream:
  - Sintassi: `nomestream.close();`
  - Es: `myin.close(); myout.close();`

Apertura di un file

- Apertura in modalità lettura (`ios::in`):
  - il file associato deve già essere presente
  - il puntatore si sposta all'inizio dello stream
- Apertura in modalità scrittura (`ios::out`):
  - il file associato se non è presente viene creato
  - il puntatore si posiziona all'inizio dello stream (sovrascrivendo il file)
- Apertura in modalità append (`ios::out|ios::app`):
  - il file associato se non è presente viene creato
  - il puntatore si posiziona alla fine dello stream

Uso di `fstream` con `argc` e `argv`

- È desiderabile poter passare i nomi dei file al programma:
  - Es: `> ./a.out pippo pluto`

$\Longrightarrow$ nomi dei file passati tramite `argc` e `argv`:

```
int main (int argc, char * argv[])
  fstream myin,myout;
  myin.open(argv[1],ios::in);
  myout.open(argv[2],ios::out);
```

- È necessario gestire l'errore utente e la mancata apertura:

```
if (argc!=3) {
  cerr << "Usage: ./a.out <source> <target>\n";
  exit(0);
}
if (myin.fail()) {
  cerr << "Il file " << argv[1] << " non esiste\n";
  exit(0);
}
```

Allocazione dinamica di memoria

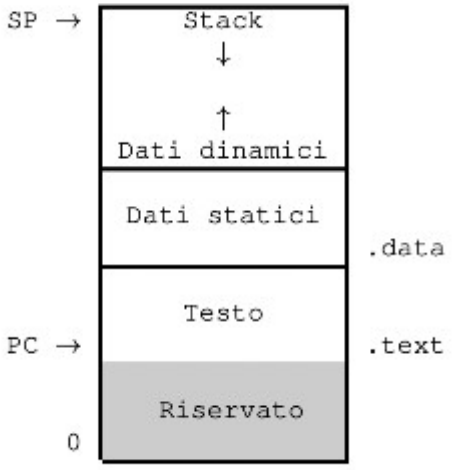
Uso Dinamico della Memoria

- L'allocazione statica obbliga a definire la struttura e la dimensione dei dati a priori (compiler time)
  - Non sempre questo è accettabile e/o conveniente
  - Esempio: dimensione di un array fissa e stabilita a priori (`int a[100];`)
- In C++ è possibile gestire la memoria anche dinamicamente, ovvero durante l'esecuzione del programma
- Memoria allocata nello store (heap), un'area esterna allo stack
- L'accesso avviene tramite puntatori
- L'allocazione/deallocazione è gestita dagli operatori **`new`** e **`delete`**

Modello di gestione della memoria per un programma (ripasso)

Area di memoria allocata ad un'esecuzione di un programma:

- Area programmi: destinata a contenere le istruzioni (in linguaggio macchina) del programma
- Area dati statici: destinata a contenere variabili globali o allocate staticamente e le costanti del programma.
- Area heap: destinata a contenere le variabili dinamiche (di dimensioni non prevedibili a tempo di compilazione) del programma.
- Area stack: destinata a contenere le variabili locali e i parametri formali delle funzioni del programma.



Allocazione: l'operatore **new**

- Sintassi:
  - **new** `tipo;`
  - **new** `tipo (valore);` (con inizializzazione del valore)
  - **new** `tipo[dimensione];` (per gli array)

  dove `dimensione` può essere un'espressione variabile e `valore` deve essere un valore costante di tipo `tipo`
- Esempio:

```
int *p, *q;
char *stringa;

p = new int;
q = new int (5); // Assegna valore 5 all'area di memoria
stringa = new char[3*i];
```

Allocazione: l'operatore **new** - II

- L'operatore **new**/**new**`[dimensione]`:
  1. alloca un'area di memoria adatta a contenere un oggetto (o `dimensione` oggetti) del tipo specificato
  2. la inizializza a `valore` (se specificato)
  3. ritorna l'indirizzo (del primo elemento) di tale area
     $\Longrightarrow$ tipicamente assegnato ad un puntatore

Deallocazione: l'operatore **delete**

- Sintassi:
  - **delete** `indirizzo;`
  - **delete**`[]` `indirizzo;` (per gli array)

  dove il valore dell'espressione `indirizzo` deve essere l'indirizzo di una celletta precedentemente allocata da una chiamata a **new**
- Esempio:

```
int *p;
char *stringa;

p = new int;
stringa = new char[30];
delete p;
delete[] stringa;
```

Deallocazione: l'operatore **delete** - II

- L'operatore **delete**/**delete**`[]` dealloca l'area di memoria precedentemente allocata a partire dall'indirizzo specificato
  - se l'indirizzo non corrisponde ad una chiamata a **new** $\Longrightarrow$ errore
  - un'area allocata da **new** [resp. **new**`[...]`] deve essere deallocata con **delete** [resp. **delete**`[]`] (altrimenti comp. non specificato)
- Al termine del programma anche la memoria allocata con **new** viene automaticamente deallocata

Deallocare un'area di memoria:

- significa che quell'area non é più "riservata"
  $\Longrightarrow$può essere ri-allocata
- non significa che il suo contenuto venga cancellato!
  $\Longrightarrow$ valore potenzialmente ancora accessibile per un po' di tempo (non noto a priori!!!)
  $\Longrightarrow$ facile non accorgersi di situazione di errore!!!

Durata di un'Allocazione Dinamica

- Un oggetto creato dinamicamente resta allocato finché:
  - non viene esplicitamente deallocato con l'operatore **`delete`**, oppure
  - il programma non termina
- La memoria allocata con **`new`** non esplicitamente deallocata con **`delete`**, può risultare non più disponibile per altri programmi
  $\Longrightarrow$ spreco di memoria (memory leak)
  $\Longrightarrow$ degrado delle prestazioni della macchina

Regola aurea

In un programma, si deve sempre esplicitamente deallocare tutto quello che si è allocato dinamicamente non appena non serve più.

Gestione dinamica della memoria: pro e contro

- Pro:
  - Gestione efficiente della memoria: alloca solo lo spazio necessario
  - Permette la creazione di strutture dati dinamiche (liste, alberi, ...)
- Contro:
  - Molto più difficile da gestire
  - Facile introdurre errori e/o memory leaks

Nota

- Esistono strumenti a supporto dell'identificazione di memory leaks:
  - Open source (e.g. `valgrind`, `gperftool`, `-fsanitize=...`)
  - ... e commerciali (e.g. `Parasoft Insure++`, `IBM Rational Purify`)

Allocazione dinamica di Array

- Consente di creare a run-time array di dimensioni diverse a seconda della necessità
- Un array dinamico è un puntatore al primo elemento della sequenza di celle

```
int n;
cin >> n;
int *a = new int[n]; //allocazione dell'array
for (int i=0; i<n; i++) {
  cout << endl << i+1 << ":␣";
  cin >> a[i]; };
delete[] a;          //deallocazione dell'array
```

Allocazione dinamica di Stringhe

- Consente di creare a run-time stringhe di dimensioni diverse
- Una stringa dinamica è un puntatore al primo elemento della sequenza di caratteri, terminata da `'\0'`
- L'I/O di una stringa dinamica è gestita automaticamente dagli operatori `>>`, `<<`
- Tutte le primitive su stringhe in `<cstring>` applicano anche alle stringhe dinamiche

```
char * sc, *sb = new char [20];
cin >> sb;
sc = new char[strlen(sb)+1];
strcpy(sc,sb);
cout << sc;
```

Fallimento di `new`

- L'esecuzione di una **new** può non andare a buon fine
  (memoria destinata al programma esaurita)
  - In tal caso lo standard C++ prevede che, se non diversamente specificato, **new** richieda al s.o. di abortire il programma.
- Soluzione: usare "**new (nothrow)**"
  - Con l'opzione "**nothrow**", **new** non abortisce ma restituisce "**NULL**" in caso di impossibilità ad allocare la memoria richiesta.
  - Esempio:

```
char *p = new (nothrow) char[mymax];
...
if (p!=NULL)...
```

Restituzione di Array II[1]

Una funzione può restituire un array se allocato dinamicamente al suo interno.

```
int *times(int a[], ...) {
  int * b = new int[10];
  (...)
  return b;
}

int v[10] = {1,2,3,4,5,6,7,8,9,10};
int * w = times(v,...);
```

Responsabilità di allocazione/deallocazione dinamica

Quando si usa allocazione dinamica di un dato (e.g. di un array) che viene passato tra più di una funzione, il programmatore deve:

- decidere quale funzione ha la responsabilità di allocare il dato
  - rischio di mancanza di allocazione $\Longrightarrow$segmentation fault
  - rischio di allocazioni multiple $\Longrightarrow$memory leak
- decidere quale funzione ha la responsabilità di deallocarlo
  - rischio di mancanza di deallocazione $\Longrightarrow$memory leak
  - rischio di deallocazioni multiple $\Longrightarrow$segmentation fault
- adeguare il passaggio di parametri delle funzioni in tal senso.
  - rischio di mancanza di allocazione $\Longrightarrow$segmentation fault

Nota importante

È fondamentale concordare preventivamente la responsabilità dell'allocazione e deallocazione quando il codice è sviluppato in team!

Allocazione dinamica di un array multidimensionale

- In C++ non è possibile allocare direttamente un array multi-dimensionale in modo dinamico

$\Longrightarrow$ array multidimensionali e puntatori sono oggetti incompatibili.

```
int *  MAT1 = new int[2][3]; // ERRORE
int ** MAT2 = new int[2][3]; // ERRORE
```

- "`new int[2][3]`" restituisce l'indirizzo di 2 oggetti consecutivi di tipo "`int[3]`", incompatibili sia con "`int *`" che con "`int **`"

Array dinamici multidimensionali

- In C++ si possono definire array dinamici multidimensionali come array dinamici di array dinamici ...
- Tipo base: puntatore di puntatore ...
- Gli operatori "[]" funzionano come nel caso statico
  - `MAT[i]` equivalente a `*(MAT+i)`,
  - `MAT[i][j]` equivalente a `*((*(MAT+i))+j)`,
- L'allocazione richiede un ciclo (o più)

```
int ** M;                       // puntatore a puntatori a int
M = new int *[dim1];            // array dinamico di puntatori
for (int i=0;i<dim1;i++)
  M[i] = new int[dim2];         // allocazione di ciascun array
```

```
void print_matrix_dim(float ** a, ...) {...}
void print_matrix_sta(float a[][d2a], ...)  {...}

float A[d1a][d2a] = {{1,2,3},{4,5,6}};
float ** B;
B = read_matrix(d1b,d2b);
// B = A;                           // errore
// print_matrix_dim (A, d1a, d2a); // errore
print_matrix_dim(B, d1b, d2b);
print_matrix_sta(A, d1a, d2a);
// print_matrix_sta(B, d1b, d2b); // errore
```

Struct

Le Strutture

- Una struttura è una collezione ordinata di elementi non omogenei
  - Gli elementi sono detti membri o campi
  - Ciascun campo ha uno specifico tipo, nome e valore
- Permette di definire nuovi tipi di oggetti aggregati
  - La struttura può essere utilizzata come un oggetto unico
  - I campi possono essere utilizzati singolarmente
- Ciascun campo può essere a sua volta un tipo struttura

Definizione di un tipo `struct`

- Viene definito un nuovo tipo aggregato
- Sintassi:

```
struct new_struct_id {
 tipo1 campo1;
 ...
 tipoN campoN;
};
new_struct_id var_id;
```

- Esempio:

```
struct complex { // definizione del tipo "complex"
 double re;  // campo "reale"
 double im;  // campo "immaginario"
};

complex c,c1; // definizione di variabili di tipo "complex"
```

Alcuni esempi di strutture annidate

```
struct data {
  int giorno, mese, anno;
};

struct persona {// struttura annidata
  char nome[25], cognome[25];
  char comune_nascita[25];
  data data_nascita;
  enum { F, M } sesso;
};

struct studente { // struttura ulteriormente annidata
  persona generalita;
  char matricola[10];
  int anno_iscrizione;
};
```

Inizializzazione di variabili di tipo `struct`

- Una variabile di tipo **`struct`** viene inizializzata con liste ordinate dei valori dei campi rispettivi
  - Devono combaciare per ordine e tipo
  - Eventuali valori mancanti vengono iniziati allo zero del tipo

```
struct data {
  int giorno, mese, anno;
};
struct persona {
  char nome[25], cognome[25];
  char comune_nascita[25];
  data data_nascita;
  enum { F, M } sesso;
};
persona x = {"Paolo", "Rossi", "Trento",
             {21,10,1980}, M };
```

Accesso ai campi di una `struct`

- Se `s` è una **`struct`** e `field` è un identificatore di un suo campo, allora `s.field` denota il campo della **`struct`**
  - `s.field` è un'espressione dotata di indirizzo!

  $\Longrightarrow$ può essere letta con `>>`, assegnata, passata per riferimento, ecc.
- Se `ps` è un puntatore ad una struttura avente `field` come campo, allora è possibile scrivere `ps->field` al posto di `(*ps).field`
  - zucchero sintattico
  - uso molto frequente

Esempio

```
struct complex { double re, im; };
complex c; complex *pc = &c;
c.re = 2.5; pc->im = 3;
cin >> c.re >> c.im;
swap(c.re,c.im);
```

Assegnazione di Strutture

- A differenza degli array, l'assegnazione tra **`struct`** è definita
- L'assegnazione di **`struct`** avviene per valore
  - Vengono copiati tutti i valori dei membri
  - Se un campo è un array statico, viene copiato per intero!

$\Longrightarrow$ La copia di **`struct`** può essere computazionalmente onerosa!

```
persona x,y = {"Paolo", "Rossi", "Trento",
               {21,10,1980}, M };

x=y; // vengono copiate tutte le stringhe!
```

Passaggio di Strutture a Funzioni

- A differenza degli array, le strutture:
  - Possono essere passate per valore ad una funzione
  - Possono essere restituire da una funzione tramite **`return`**
- Entrambe le operazioni comportano una copia dei valori dei membri (array compresi!)
- $\Longrightarrow$ Entrambe le operazioni possono essere computazionalmente onerose!
- $\Longrightarrow$ Quando possibile, è preferibile utilizzare passaggio per riferimento (con **`const`**)

```
void stampa_persona (persona p) {...}
void stampa_persona1 (const persona & p) {...}

persona x,y = {"Paolo", "Rossi", "Trento",
               {21,10,1980}, M };
stampa_persona(y);  // viene fatta una copia
stampa_persona1(y); // non viene fatta alcuna copia
```

Assegnazione di array statici tramite **`struct`**

Per gestire gli array statici in modo che possano essere copiati, si può "incapsulare" un tipo array come membro di una **`struct`**

```
struct int_array { int ia[3]; };
int_array sa, sb;
sa.ia[0]=1;
sa.ia[1]=2;
sa.ia[2]=3;
sb = sa; // l'array viene copiato!
```

Assegnazione di array dinamici tramite `struct`

Nel caso di array dinamici, viene copiato solo il puntatore

```
struct int_array {   int * ia; };
sa.ia = new int[3];
sb = sa; // viene copiato il puntatore,
         // sa.ia e sb.ia sono lo stesso array!
```

Strutture ricorsive

- La seguente definizione non è lecita:

```
struct S {
  int value;
  S next; //definizione circolare!
};
```

- La seguente definizione è lecita:

```
struct S {
  int value;
  S *next;
};
```

- Ogni puntatore occupa lo stesso spazio di memoria indipendentemente dal suo tipo.
- Molto importante per strutture dati dinamiche!

- La seguente definizione non è lecita:

```
struct S1
{ int value;
  S2 *next;  }; //S2 ancora indefinito
struct S2
{ int value;
  S1 *next; };
```

- La seguente definizione è lecita:

```
struct S2;  // dichiarazione di S2
struct S1
{ int value;
  S2 *next; }; // Ok!
struct S2   // definizione di S2
{ int value;
  S1 *next;};
```

Uso di Array Ordinati di Strutture

- È frequente il dover gestire archivi ordinati di oggetti complessi (ES: persone, articoli, libri, ecc.)
  - Elemento base dei sistemi informativi
    (ES: archivi, inventari, rubriche, anagrafi, ecc.)
  - Ordinamento usa qualche campo specifico (chiave)
- Tipicamente utilizzate strutture di dati dinamiche ad hoc
  (ES: alberi di ricerca binaria)
- Esempio di archivio semplificato: array ordinato di persone

Esempio 1: Array Ordinato di Persone

- Array ordinato di strutture "`persona`":
  ```
  persona persone [NmaxPers];
  ```
- Ordinamento e ricerca usano `strcmp` sul campo `cognome`:
  ```
  if (strcmp(p[i].cognome,cognome) < 0) ...
  ```
- Array ordinato di **`struct`**, bubblesort, ricerca binaria:
  { `STRUCT/persone.cc` }
- Problema: ogni swap effettua 3 copie tra **`struct`**!
  $\Longrightarrow$ molto inefficiente!

Esempio 2: Array Ordinato di Puntatori a Persone

- Array ordinato di puntatori a strutture "`persona`":
  ```
  persona * persone [NmaxPers];
  ```
- Ordinamento e ricerca usano `strcmp` sul campo `cognome`:
  ```
  if (strcmp(p[i]->cognome,cognome) < 0) ...
  ```
- Array ordinato di puntatori a **`struct`**, bubblesort, ricerca binaria:
  { `STRUCT/persone2.cc` }
- Importante: ogni swap effettua 3 copie tra puntatori
  $\Longrightarrow$ efficiente!

Es. 3: Doppio Array Ordinato di Puntatori a Persone

- Richiesta: poter effettuare ricerca sia per nome che per cognome.
- Idea: 2 array di puntatori a strutture "`persona`", ordinati rispettivamente per nome e cognome

```
persona * nomi [NmaxPers];
persona * cognomi [NmaxPers];
```

- Ordinamento e ricerca usano `strcmp` sul campo `cognome` e `nome` rispettivamente:

```
if (strcmp(p[i]->cognome,cognome) < 0) ...
if (strcmp(p[i]->nome,nome) < 0) ...
```

- Array ordinato di puntatori a **`struct`**, bubblesort, ricerca binaria:
  { `STRUCT/persone3.cc` }
- Importante: Le **`struct`** sono condivise tra i due array, ogni swap effettua 3 copie tra puntatori
  $\Longrightarrow$ efficiente!

Organizzazione di un programma su più file

Scope, Visibilità e Durata di una definizione

La definizione di un oggetto (variabile, costante, tipo, funzione) ha tre caratteristiche:

- Scope o ambito
- Visibilità
- Durata

Scope di una definizione

È la porzione di codice in cui è attiva una definizione

- Scope globale: definizione attiva a livello di file
- Scope locale: definizione attiva localmente
  - ad una funzione
  - ad un blocco di istruzioni

```
const float pi=3.1415;      // scope globale
int x;                      // scope globale
int f(int a, double x);     // scope locale
{  int c;                   // scope locale
  ...}
int main()
{  char pi;                 // scope locale
  ...}
```

Visibilità di una definizione

Stabilisce quali oggetti definiti sono visibili da una punto del codice

- In caso di funzioni:
  - una definizione globale è visibile a livello locale, ma non viceversa
  - una definizione omonima locale maschera una definizione globale
- in caso di blocchi annidati
  - una definizione esterna è visibile a livello interno, ma non viceversa
  - una definizione omonima interna maschera una definizione esterna

```
1. const float pi=3.1415; // sono visibili:
2. int x;                 // pi(1)
3. int f(int a, double x)
4. {  int c;              // pi(1), a(3), x(3)
5.    ...}                // pi(1), a(3), x(3), c(4)
6. int main()
7. {  char pi;            // x(2), f(3),
8.   ... }                // x(2), f(3), pi(7)
```

Durata di una definizione

Stabilisce il periodo in cui l'oggetto definito rimane allocato in memoria

- Globale o Statico: oggetto globale o dichiarato con **`static`**
  - dura fino alla fine dell'esecuzione del programma
  - memorizzato nell'area dati statici
- Locale o automatico: oggetti locali a un blocco o funzione
  - hanno sempre scope locale
  - durano solo il periodo di tempo necessario ad eseguire il blocco o funzione in cui sono definiti
  - memorizzato nell'area stack
- Dinamico: oggetti allocati e deallocati da **`new`**/**`delete`**
  - hanno scope determinato dalla raggiungibilità dell'indirizzo.
  - durata gestita dalle chiamate a **`new`** e **`delete`**: durano fino alla deallocazione con **`delete`** o alla fine del programma
  - dimensione non prevedibile a tempo di compilazione
  - memorizzati nell'area heap

Lo specificatore `static`

Lo specificatore **`static`** applicato ad una variabile locale forza la durata della variabile oltre la durata della funzione dove è definita

- la variabile è allocata nell'area dati statici
- un'eventuale inizializzazione nella dichiarazione viene eseguita una sola volta all'atto dell'inizializzazione del programma
- il valore della variabile viene "ricordato" da una chiamata all'altra della funzione
- potenziali sorgenti di errori $\Longrightarrow$ vanno usate con molta cautela!

Nota

Lo specificatore **`static`** applicato ad un oggetto di scope globale (es. funzioni, variabili, costanti globali) ha l'effetto di restringere la visibilità dell'oggetto al solo file in cui occorre la definizione

- concetto molto importante nella programmazione su più file
  (vedi slide successive)

Lo specificatore `extern`

Lo specificatore **`extern`** consente di dichiarare e poi utilizzare in un file oggetti (globali) che sono definiti in un altro file

- consente al compilatore di
  - verificare la coerenza delle espressioni contenenti tali oggetti
  - stabilire le dimensioni delle corrispondenti aree di memoria
- l'oggetto dichiarato deve essere definito in un altro file
- il linker associa gli oggetti dichiarati alle corrispondenti definizioni

Dichiarazione vs Definizione

Nota

- Un oggetto può essere dichiarato quante volte si vuole ...
  ... ma può essere definito una volta sola
- Un oggetto dichiarato più volte deve essere dichiarato sempre nello stesso modo
- Ogni definizione è anche una implicita dichiarazione

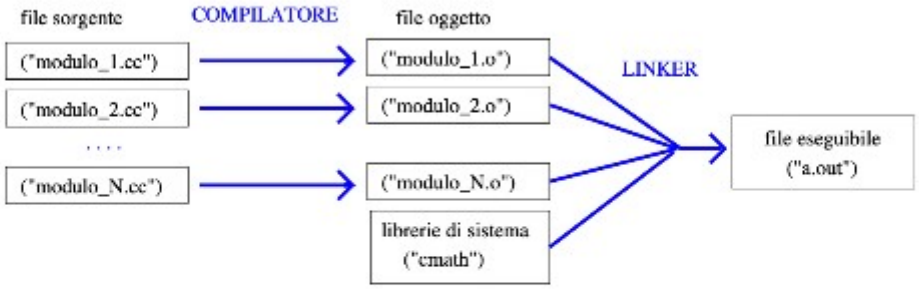
Organizzazione di un programma su file multipli - II

- Ogni file "`.cc`" che utilizzi funzioni/tipi/costanti/variabili globali definiti in `modulo_i` deve inizialmente contenere l'istruzione: **`#include`** `"modulo_i.h"`
- `modulo_i.h` contiene gli header delle funzioni di `modulo_i`
  - può contenere definizioni di tipo, costanti, variabli globali, ecc.
  - per evitare di venire caricato ripetutamente deve utilizzare guardie di compilazione:

```
#ifndef MODULO_I_H          // Esiste anche direttiva
#define MODULO_I_H          // #pragma once
...                         // Non e' standard e non e' robusto
...                         // come approccio (e.g. link,
#endif                      // copie di file, ...)
```

- `modulo_i.cc` contiene le definizioni delle funzioni di `modulo_i`
  - contenere l'istruzione: **`#include`** `"modulo_i.h"`
  - può contenere funzioni ausiliarie inaccessibili all'esterno (**`static`**)

Compilazione (su più files) - recap



- File sorgente tradotti nei rispettivi file oggetto uno alla volta
- File oggetto collegato (linked) a librerie di sistema dal linker, generano un file eseguibile (default `a.out`)
  - Es: `g++ modulo_1.o modulo_2.o ... modulo_N.o`
- Compilazione e linking possibile in un unica istruzione
  - Es: `g++ modulo_1.cc modulo_2.cc ... modulo_N.cc`

Schema: Programma su un solo file

disney.cc

```
#include <iostream>
int pluto() {...};          // funzione ausiliaria
                            // non chiamata dal main()
void topolino() { ... };
void paperino() { ... };

int main() {
  ...
  switch(scelta) {
    case 1: topolino(); break;
    case 2: paperino(); break;
    ...
}
```

Schema: Programma su file multipli - I

disney.h

```
#ifndef DISNEY_H
#define DISNEY_H
void topolino();
void paperino();
#endif
```

disney.cc

```
#include <iostream>
#include "disney.h"
static int pluto() {...};  // funzione ausiliaria non chiamata
                           // dal main().
                           // Header non in disney.h!
void topolino() { ... };
void paperino() { ... };
```

Schema: Programma su file multipli - II

disney_main.cc

```
#include <iostream>
#include "disney.h"

int main() {
  ...
  switch(scelta) {
    case 1: topolino(); break;
    case 2: paperino(); break;
    ...
}
```

Bad programming practices

Cosa non fare e come non organizzare un programma su file multipli

- Includere i file ".h" (e compilare tutti i file, ".h" inclusi)
- NON includere i file ".cc" (e compilare solo il file chiamante) !!!
  - impedirebbe uso multiplo (non hanno guardie di compilazione)
  - romperebbe la modularità
  - impedirebbe la compilazione separata
- $\Longrightarrow$ naturale sorgente di errori, evitare tassativamente

disney_main.cc

```
#include "disney.cc" // NNOO!!!!!
...
```

disney_main.cc

```
#include "disney.h"  // SI!
...
```

Strutture dati fondamentali (liste, stack, code)

Liste

Introduzione

- Quando dobbiamo scandire una collezione di oggetti in modo sequenziale e non sequenziale, un modo conveniente per rappresentarli è quello di organizzare gli oggetti in un array.
- Esempi:
  - `{1, 2, -3, 5, -10}` è una sequenza di interi.
  - `{'a', 'd', '1', 'F'}` è una sequenza di caratteri.
- Una soluzione alternativa all'uso di array per rappresentare collezioni di oggetti quando l'accesso non sequenziale non è un requisito, consiste nell'uso delle cosiddette liste concatenate.
- In una lista concatenata i vari elementi che compongono la sequenza di dati sono rappresentati in zone di memoria che possono anche essere distanti fra loro (al contrario degli array, in cui gli elementi sono consecutivi).
- In una lista concatenata, ogni elemento contiene informazioni necessarie per accedere all'elemento successivo.

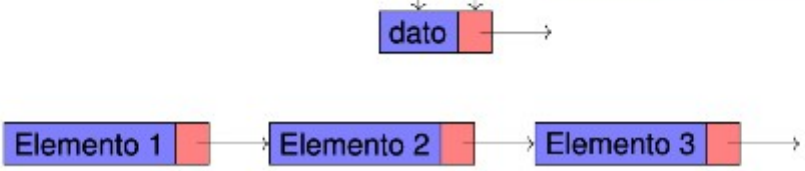
Liste concatenate

- Una lista concatenata è un insieme di oggetti, dove ogni oggetto è inserito in un nodo contenente anche un link ad un altro nodo.

```
// Lista concatenata di interi
struct nodo {
  int dato;
  nodo * next;
}
```

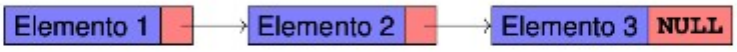
Campo contenente informazioni del nodo

Puntatore al nodo successivo



Liste concatenate (II)

- Per il nodo finale si possono adottare diverse convenzioni:
  - Punta ad un link nullo che non punta a nessun nodo (e.g. `<val> = NULL`)



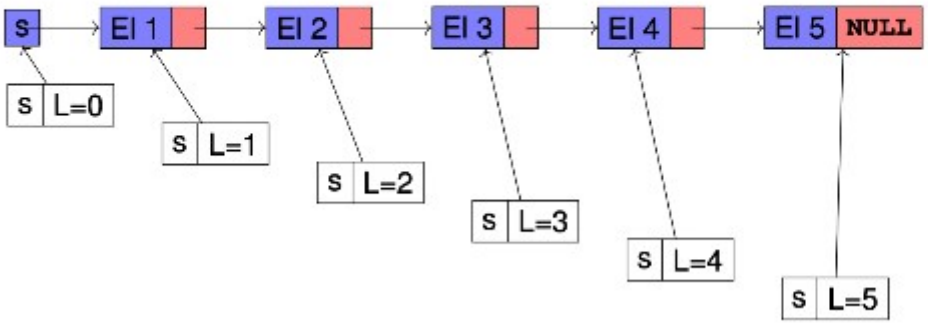
  - Punta ad un nodo fittizio che non contiene alcun nodo (e.g. `<val> = (nodo *)300`)



  - Punta indietro al primo elemento della lista, creando una lista circolare



Calcolo lunghezza di una lista



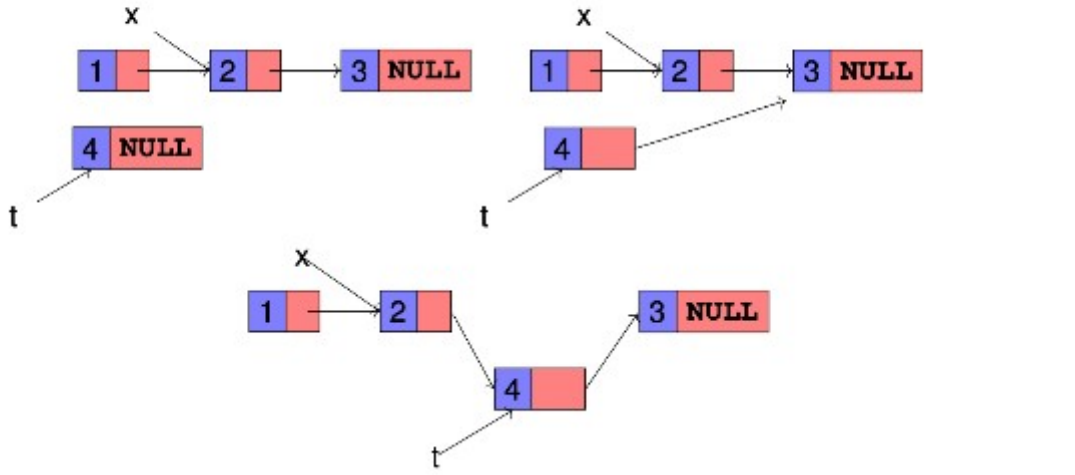
Calcolo lunghezza di una lista (II)

```
// lista con terminatore NULL
int length (nodo * s) {
  int l = 0;
  for( ; s != NULL; s = s->next) l++;
  return l;
}
```

```
// lista circolare, x primo elemento
int length (nodo * s, nodo * x) {
  int l = 0;
  if (s != NULL) {
    l = 1;
    for( s = s->next; s != x; s = s->next) l++;
  }
  return l;
}
```

Inserimento di un elemento

- Per inserire un nodo `t` in una lista concatenata nella posizione successiva a quella occupata da un dato nodo `x`, poniamo `t->next` a `x->next`, e quindi `x->next` a `t`.



Inserimento di un elemento (II)



Inserimento di un elemento (II)

```
int main () {
  nodo * x = new nodo;
  cout << "Inserire numero: ";
  cin >> x->dato;
  x->next = NULL;
  for (int i = 0; i < 10; i++) {
    nodo * t = new nodo;
    cout << "Inserire un numero: ";
    cin >> t->dato;
    t->next = NULL;
    insert_node(x, t);
  }
  for (nodo *s =x; s != NULL; s=s->next)
     cout << "valore = " << s->dato << endl;
}
```

Allocazione di un nodo per memorizzare primo elemento

Allocazione di un nuovo nodo per memorizzare i-esimo elemento

Campo next di t inizializzato a NULL

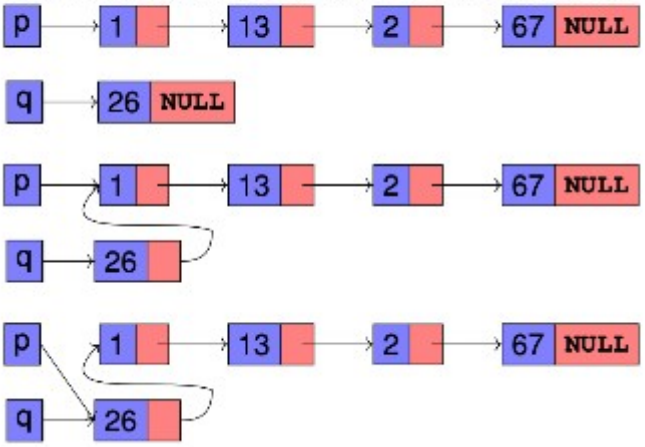
Inserzione del nuovo elemento t successivamente al nodo iniziale x

Variabile temporanea per scorrere lista

Manca deallocazione della lista!!!

Inserimento di un elemento in testa

- Si vuole inserire un nuovo elemento 26 in testa alla lista!




```
node *q = new nodo;
q->dato = 26;
```



```
q->next = p;
```



```
p = q;
```

Inserimento di un elemento in testa (II)

- Se dobbiamo inserire un elemento in testa della lista:
  - `void insert_first(nodo * s, int v);`
  - `void insert_first(nodo * &s, int v);`
  - `nodo * insert_first(nodo * s, int v);`
- La seconda e la terza sono le uniche possibili dichiarazioni corrette.
  - Nella seconda side effect sull'argomento.
  - Nella terza lista nuova ritornata dalla funzione.

Inserimento di un elemento in testa (III)

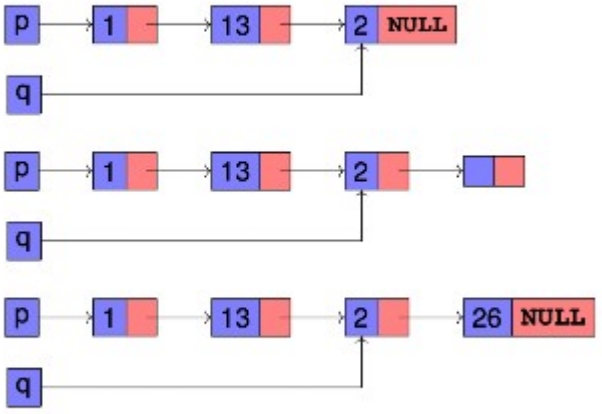
```
void insert_first(node * s, int v) {
  node * n = new node;
  n->dato = v;
  n->next = s;
  s = n;
}
```

```
void insert_first(node*&s, int v) {
  node * n = new node;
  n->dato = v;
  n->next = s;
  s = n;
}
```

```
node * insert_first(node*s, int v) {
  node * n = new mode;
  n->dato = v;
  n->next = s;
  return n;
}
```

Inserimento di un elemento in coda

- Si vuole inserire un nuovo elemento 26 in coda alla lista!




```
node *q = p;
while(q->next != NULL)
  q = q->next;
```

```
q->next = new node;
```

```
q->next->dato = 26;
q->next->next = NULL;
```

Nota:

Questo ragionamento è corretto solo se la lista non è vuota!!!

Inserimento di un elemento in coda (II)

```
void insert_last(nodo * & p, int n) {
  nodo * r = new nodo;
  r->dato = n;
  r->next = NULL;
  if (p != NULL) {
    node * q = p;
    while(q->next != NULL) {
      q = q->next;
    }
    q->next = r;
  }
  else {
    p = r;
  }
}
```

Allocazione del nuovo `nodo`

Se la lista non è vuota, cerco in `q` il puntatore all'ultimo elemento

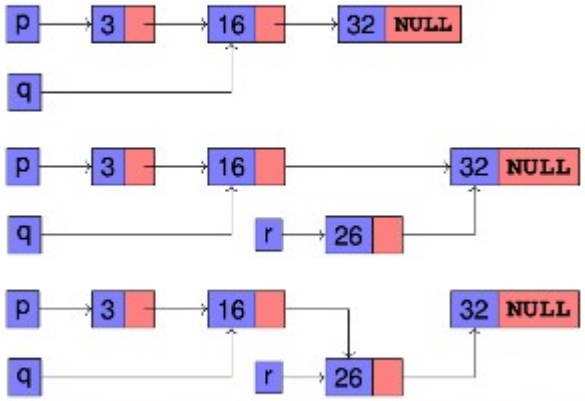
`q` è garantito essere diverso da `NULL`

Memorizzo in `q->next` il nuovo nodo `r` allocato in precedenza

Se la lista è vuota, `p` punta al nuovo nodo allocato: `p` è passato per riferimento

Inserimento di un elemento in lista ordinata

- Si vuole inserire un nuovo elemento `26` in una lista ordinata mantenendo ordinamento!




```
node *q = p;
while(q->next->dato <= 26)
  q = q->next;
```

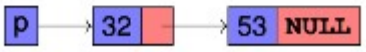
```
node *r = new node;
r->dato = 26;
r->next = q->next;
```

```
q->next = r;
```

Nota:
Devono essere considerati alcuni casi limite!!!

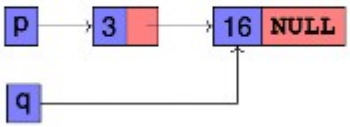
Inserimento di un elemento in lista ordinata (II)

- Primo caso limite: inserimento in testa
  - perchè la lista è vuota: `p == NULL`
  - perchè tutti gli elementi hanno un valore maggiore




```
if ((p == NULL) || (p->dato >= 26))
  insert_first(p, 26);
```

- Secondo caso limite: inserimento in coda
  - perchè tutti gli elementi hanno un valore minore



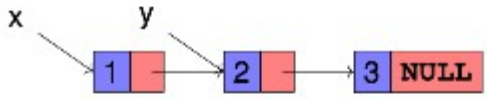

```
node *q = p;
while(q->next != NULL &&
      q->next->dato <= 26)
  q = q->next;
```

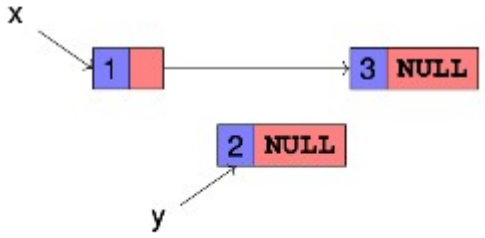
Inserimento di un elemento in lista ordinata (III)

```
void insert_order(nodo * &p, int inform){
  if ((p==NULL) || (p->dato >= inform)) {
    insert_first(p, inform);
  }
  else {
    nodo* q=p;
    while ((q->next != NULL) &&
           (q->next->dato <= inform)) {
      q=q->next;
    }
    nodo* r = new nodo;
    r->dato = inform;
    r->next = q->next;
    q->next = r;
  }
}
```

Rimozione di un elemento

- Per rimuovere un nodo `y` in una lista concatenata nella posizione successiva a quella occupata da un dato nodo `x`, cambiamo `x->next` a `y->next`.





Rimozione di un elemento (II)

```
node * remove_element(node *x) {
  node * y = x->next;
  x->next = y->next;
  y->next = NULL;
  return y;
}
```

y inizializzato a x->next

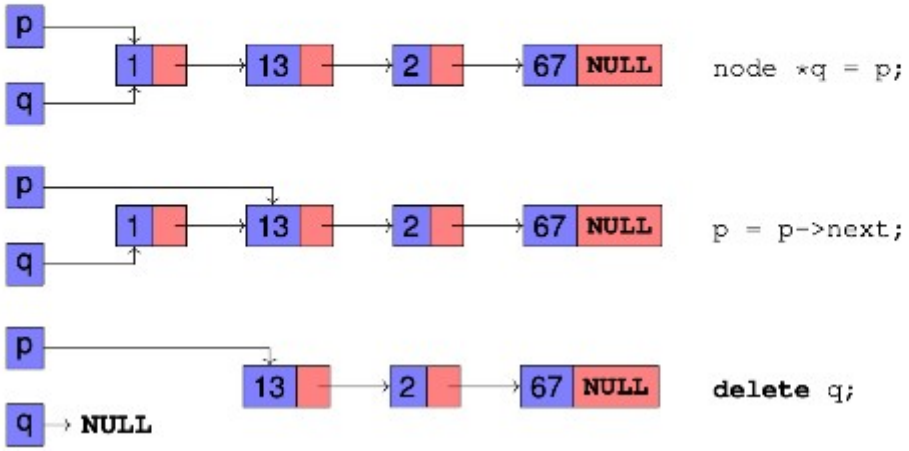
x->next punta a y->next

Assunzione che x, x->next (e quindi anche y) siano diversi da NULL

y ritornato per ad esempio essere deallocato!

Rimozione di un elemento in testa

- Si vuole eliminare primo elemento della lista!



Rimozione di un elemento in testa (II)

- Se dobbiamo rimuovere un elemento in testa della lista:
  - `void remove_first(nodo * s);`
  - `void remove_first(nodo * &s);`
  - `nodo * remove_first(nodo * s);`
- La seconda e la terza sono le uniche possibili dichiarazioni corrette.
  - Nella seconda side effect sull'argomento.
  - Nella terza lista nuova ritornata dalla funzione.

Nota:

Il nodo rimosso deve essere deallocato!!!

Rimozione di un elemento in testa (III)

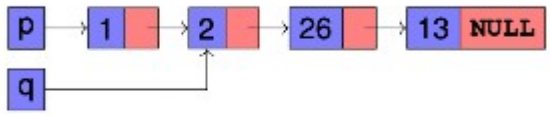
```
void remove_first(nodo * s) {
  nodo * n = s;
  if (s != NULL) {
    s = s->next;
    delete n;
  }
}
```

```
void remove_first(nodo * & s) {
  nodo * n = s;
  if (s != NULL) {
    s = s->next;
    delete n;
  }
}
```

```
nodo * remove_first(nodo * s)
{
  nodo * n = s;
  if (s != NULL) {
    s = s->next;
    delete n;
  }
  return s;
} // attenzione a come invocata
```

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un valore ed eliminarlo!



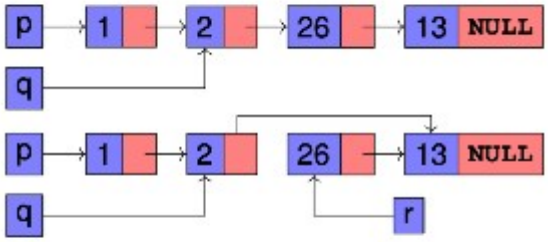

```
nodo* q=p;
```

```
while(q->next!=NULL){
  if(q->next->dato==26){


  }
  q=q->next;
}
```

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un valore ed eliminarlo!



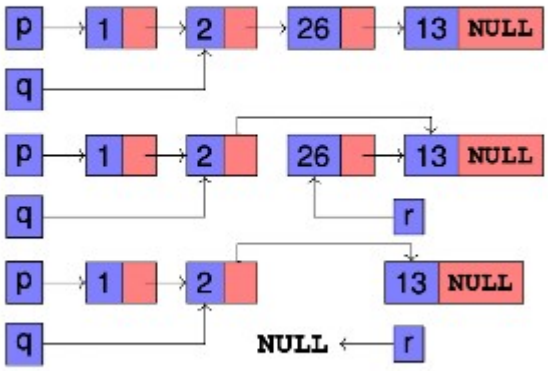

```
nodo* q=p;


while(q->next!=NULL){
  if(q->next->dato==26){
    node *r = q->next;
    q->next = q->next->next;


  }
  q=q->next;
}
```

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un dato valore ed eliminarlo!




```
nodo* q=p;


while(q->next!=NULL){
  if(q->next->dato==26){
    node *r = q->next;
    q->next = q->next->next;
    delete r;

  }
  q=q->next;
}
```

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un valore ed eliminarlo!




```
if (p != NULL) {
  nodo* q=p;

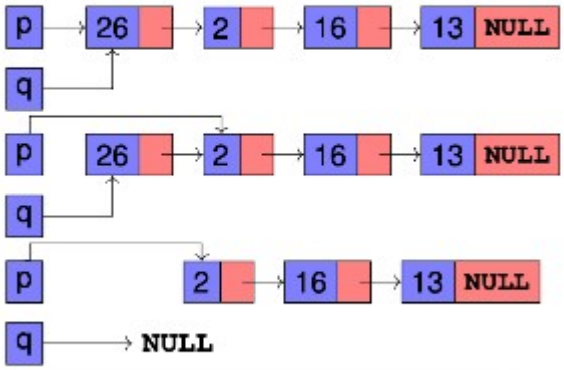

  while(q->next!=NULL){
    if(q->next->dato==26){
      node *r = q->next;
      q->next = q->next->next;
      delete r;

    }
    q=q->next;
  }
}
```

Nota:
Primo caso limite: Lista vuota!!!

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un valore ed eliminarlo!



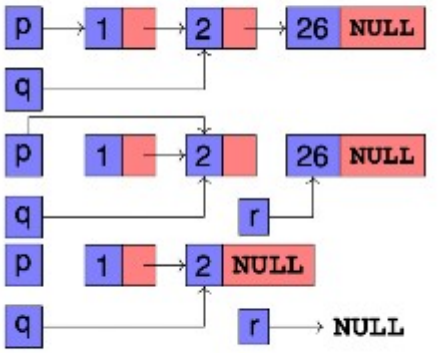

```
if (p != NULL) {
  nodo* q=p;
  if (p->dato == 26) {
    p = p->next; delete q;
  }
  while(q->next!=NULL){
    if(q->next->dato==26){
      node *r = q->next;
      q->next = q->next->next;
      delete r;

    }
    q=q->next;
  }
}
```

Nota:
Secondo caso limite: primo nodo da levare!!!

Rimozione di un elemento particolare

- Si vuole cercare un nodo che contiene nel campo `dato` un valore ed eliminarlo!



Nota:

Terzo caso limite: ultimo nodo da levare!!!

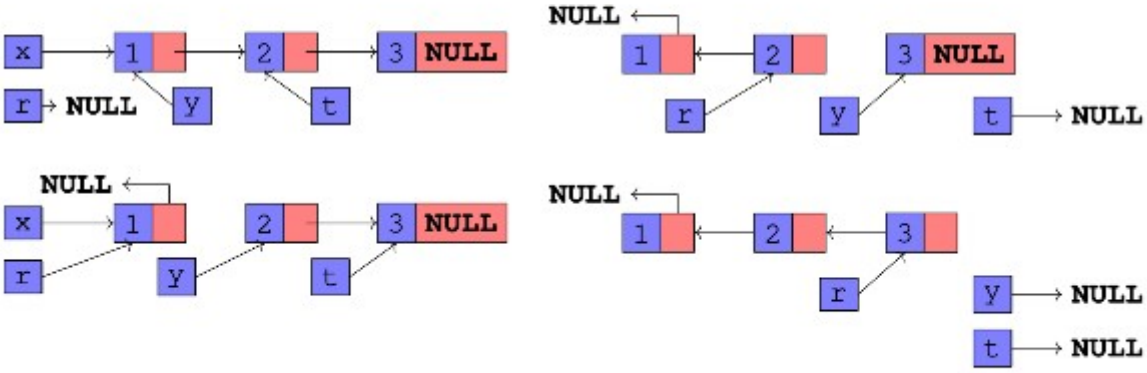
```
if (p != NULL) {
  nodo* q=p;
  if (p->dato == 26) {
    p = p->next; delete q;
  }
  while(q->next!=NULL){
    if(q->next->dato==26){
      node *r = q->next;
      q->next = q->next->next;
      delete r;
      return;
    }
    q=q->next;
  }
}
```

Rimozione di un elemento particolare (II)

```
void search_remove(nodo* &p, int val){
  if (p != NULL) {
    nodo* q = p;
    if (q->dato == val) {
      p = p->next;
      delete q;
    }
    else {
      while(q->next != NULL) {
        if (q->next->dato == val) {
          nodo* r = q->next;
          q->next = q->next->next;
          delete r;
          return;
        }
        if (q->next != NULL) {
          q=q->next;
        }
      }
    }
  }
}
```

Rovesciamento di una lista concatenata

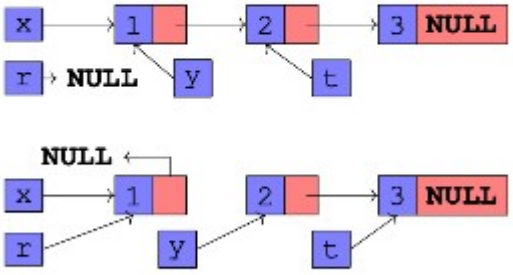
- La funzione di rovesciamento inverte i link di una lista concatenata:
  - restituisce un puntatore al nodo finale che a sua volta punta al penultimo e così via.
  - Il link del primo elemento della lista è posto a **NULL**.



Rovesciamento di una lista concatenata (II)

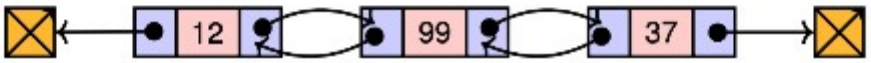
- La funzione di rovesciamento inverte i link di una lista concatenata:
  - restituisce un puntatore al nodo finale che a sua volta punta al penultimo e così via.
  - Il link del primo elemento della lista è posto a **NULL**.

```
node * reverse(node * x) {
  node * t;
  node * y = x;
  node * r = NULL;
  while ( y != NULL ) {
    t = y->next;
    y->next = r;
    r = y;
    y = t;
  }
  return r;
}
```



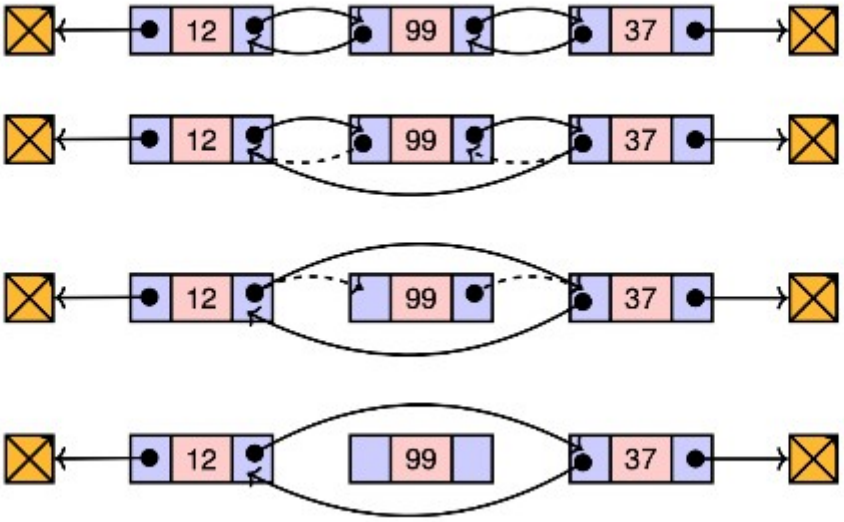
Liste doppiamente concatenate

- Sono una estensione della definizione delle liste concatenate
  - Differiscono per la presenza di un ulteriore puntatore al nodo che lo precede




```
struct node {
  int n;
  node * prev;
  node * next;
};
```

Rimozione di un nodo in una lista doppiamente concatenata



Inserimento di un nodo in una lista doppiamente concatenata

- Soluzione 1:

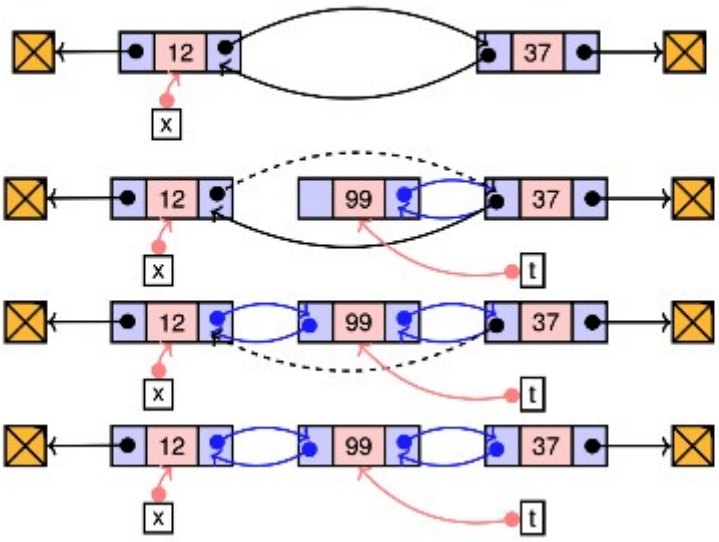
```
node * remove(node * t) {
  t->next->prev = t->prev;
  t->prev->next = t->next;
  t->next = t->prev = NULL;
  return t;
}
```

- Soluzione 2:

```
void remove(node * t) {
  t->next->prev = t->prev;
  t->prev->next = t->next;

  delete t;
}
```

Inserimento di un nodo in una lista doppiamente concatenata



Inserimento di un nodo in una lista doppiamente concatenata

```
void insert_node(node * x, node * t) {
  t->next = x->next;
  t->next->prev = t;
  t->prev = x;
  x->next = t;
}
```

Strutture dati astratti

Tipo di Dato Astratto/Abstract Data Type

Un tipo di dato astratto (TDA)/abstract data type (ADT) è un un insieme di valori e di operazioni definite su di essi in modo indipendente dalla loro implementazione

- Per definire un tipo di dato astratto occorre specificare:
  - i dati immagazzinati
  - le operazioni supportate
  - le eventuali condizioni di errore associate alle operazioni
- Per lo stesso TDA si possono avere più implementazioni
  - diversa implementazione, diverse caratteristiche computazionali (efficienza, uso di memoria, ecc.)
  - stessa interfaccia (stessi header di funzioni, riportati in un file `.h`)
    $\Longrightarrow$ implementazioni interscambiabili in un programma
- È spesso desiderabile nascondere l'implementazione di un TDA (information hiding): solo i file `.h` e `.o` disponibili

N.B.: La nozione di TDA è la base della programmazione ad oggetti.

Pile array(stack)

Le Pile (Stack)

- Una pila è una collezione di dati omogenei (e.g., puntatori a struct) in cui gli elementi sono gestiti in modo LIFO (Last In First Out)
  - Viene visualizzato/estratto l'elemento inserito più recentemente
  - Es: una scatola alta e stretta contenente documenti
- Operazioni tipiche definite su una pila di oggetti di tipo `T`:
  - `init()`/`deinit()`: inizializza/deinizializza la pila
  - `push(T)`: inserisce elemento sulla pila; fallisce se piena
  - `pop()`: estrae l'ultimo elemento inserito (senza visualizzarlo); fallisce se vuota
  - `top(T &)`: ritorna l'ultimo elemento inserito (senza estrarlo); fallisce se vuota
- Varianti:
  - `pop()` e `top(T &)` fuse in un'unica operazione `pop(T &)`
  - talvolta disponibili anche `print()`
  - [`deinit()` non sempre presente]

Nota importante

In tutte le possibili implementazioni di una pila, le operazioni `push(T)`, `pop()`, `top(T &)` devono richiedere un numero costante di passi computazionali, indipendente dal numero di elementi contenuti nella pila!

Implementazione di una pila mediante array



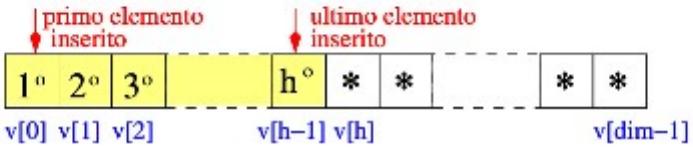
- Dati: un intero `h` e un array `v` di `dim` elementi di tipo `T`
  - `v` allocato staticamente o dinamicamente
  - `h` indice del prossimo elemento da inserire (inizialmente 0)

  $\Longrightarrow$ numero di elementi contenuti nella pila: `h`
  - pila vuota: `h==0`
  - pila piena: `h==dim`

  $\Longrightarrow$ massimo numero di elementi contenuti nella pila: `dim`

N.B.: `dim` elementi sempre allocati.

Implementazione di una pila mediante array II



- Funzionalità:
  - `init()`: pone `h=0` (alloca `v` se allocazione dinamica)
  - `push(T)`: inserisce l'elemento in `v[h]`, incrementa `h`
  - `pop()`: decrementa `h`
  - `top(T &)`: restituisce `v[h-1]`
  - `deinit()`: dealloca `v` se allocazione dinamica

Code array(queue)

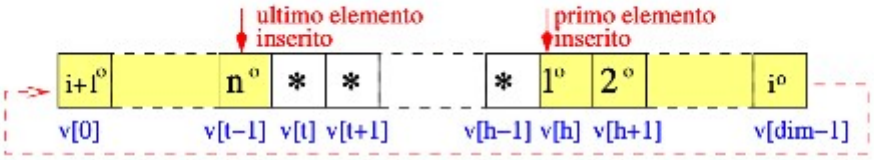
Le Code (Queue)

- Una coda è una collezione di dati omogenei in cui gli elementi sono gestiti in modo FIFO (First In First Out)
  - Viene visualizzato/estratto l'elemento inserito meno recentemente
  - Es: una coda ad uno sportello
- Operazioni tipiche definite su una coda di oggetti di tipo `T`:
  - `init()`/`deinit()`: inizializza/deinizializza la coda
  - `enqueue(T)`: inserisce elemento sulla coda; fallisce se piena
  - `dequeue()`: estrae il primo elemento inserito (senza visualizzarlo); fallisce se vuota
  - `first(T &)`: ritorna il primo elemento inserito (senza estrarlo); fallisce se vuota
- Varianti:
  - `dequeue()` e `first(T &)` fuse in un'unica operazione `dequeue(T &)`
  - talvolta disponibili anche `print()`
  - [`deinit()` non sempre presente]

Nota importante

In tutte le possibili implementazioni di una coda, le operazioni `enqueue(T)`, `dequeue()`, `first(T &)` devono richiedere un numero costante di passi computazionali, indipendente dal numero di elementi contenuti nella coda!

Implementazione di una coda mediante array



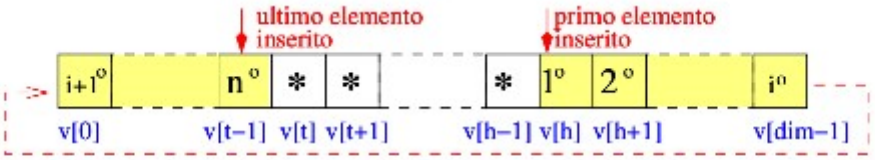
- Idea: buffer circolare: `succ(i)==(i+1)%dim`
- Dati: due interi `h`, `t` e un array `v` di `dim` elementi di tipo `T`
  - `v` allocato staticamente o dinamicamente
  - `h` indice del più vecchio elemento inserito (inizialmente 0)
  - `t` indice del prossimo elemento da inserire (inizialmente 0)

  ⟹ num. di elementi contenuti nella coda: `n=(t>=h ? t-h : t-h+dim)`
  - coda vuota: `t==h`
  - coda piena: `succ(t)==h`

  ⟹ massimo numero di elementi contenuti nella coda: `dim-1`

N.B.: `dim` elementi sempre allocati.

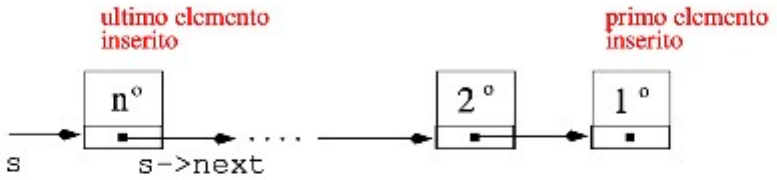
Implementazione di una coda mediante array II



- Funzionalità:
  - `init()`: pone `h=t=0` (alloca `v` se allocazione dinamica)
  - `enqueue(T)`: inserisce l'elemento in `v[t]`, "incrementa" `t` (`t=succ(t)`)
  - `dequeue()`: "incrementa" `h`
  - `first(T &)`: restituisce `v[h]`
  - `deinit()`: dealloca `v` se allocazione dinamica

Pile lista concatenata

Implementazione di una pila come lista concatenata

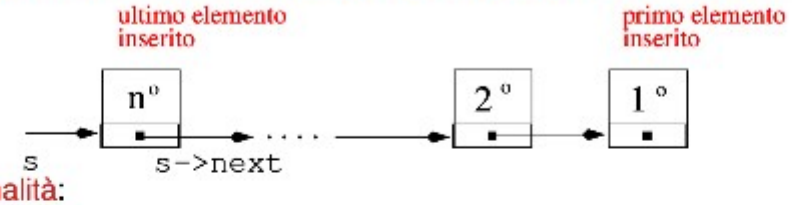


- Dati: una lista concatenata `s` di `n` elementi
  - `s` punta all'ultimo elemento inserito nella pila (inizialmente NULL)
  - [Opzionalmente un intero `n` con il numero di elementi nella lista]
  - l'ultimo elemento della lista contiene il primo elemento inserito.
  - pila vuota: `s==NULL`
  - pila piena: `out of memory`

  $\Longrightarrow$ numero di elementi contenuti nella pila limitato dalla memoria

N.B.: allocati solo gli `n` nodi necessari a contenere gli elementi

Implementazione di una pila come lista concatenata II



- Funzionalità:
  - `init()`: pone `s=NULL`
  - `push(T)`:
    1. alloca un nuovo nodo ad un puntatore `tmp`
    2. copia l'elemento in `tmp->value`
    3. assegna `tmp->next=s`, e `s=tmp`
  - `pop()`:
    1. fa puntare un nuovo puntatore `first` al primo nodo: `first=s`
    2. `s` aggira il primo nodo: `s=s->next`
    3. dealloca (l'ex) primo nodo: **`delete`** `first`
  - `top(T &)`: restituisce `s->value`
  - `deinit()`: ripete `pop()` finché la pila non è vuota

Code lista concatenata

Implementazione di una coda come lista concatenata



- Dati: una lista concatenata `h` di `n` elementi di tipo `T`, un puntatore `t` all'ultimo elemento
  - `h` punta al primo elemento inserito nella coda (inizialmente NULL)
  - `t` punta all'ultimo elemento inserito nella coda
  - [Opzionalmente un intero `n` con il numero di elementi nella lista]
  - coda vuota: `h==NULL`
  - coda piena: `out of memory`

  ⟹ numero di elementi contenuti nella coda limitato dalla memoria

N.B.: allocati solo gli `n` nodi necessari a contenere gli elementi

Implementazione di una coda come lista conc. II



- Funzionalità:
  - `init()`: pone `h=`**`NULL`**
  - `enqueue(T)`:
    1. alloca un nuovo nodo ad un puntatore `tmp`
    2. copia l'elemento in `tmp->value` e pone `tmp->next=`**`NULL`**
    3. (se coda non vuota) assegna `t->next=tmp`, e `t=tmp`
       (se coda vuota) assegna `h=tmp`, e `t=tmp`
  - `dequeue()`: come `pop()` della pila con il puntatore `h`
  - `first(T &)`: come `top()` della pila con il puntatore `h`
  - `deinit()`: ripete `dequeue()` finché la coda non è vuota

Alberi binari (tree)

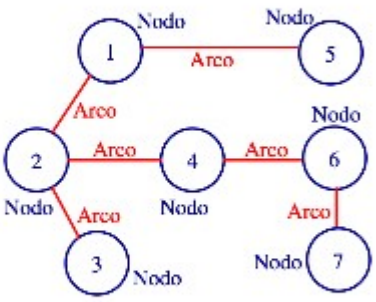
Alberi

- Gli alberi sono una struttura matematica che gioca un ruolo molto importante nella progettazione e nell'analisi di algoritmi:
  - Sono spesso utilizzati per descrivere proprietà dinamiche degli algoritmi.
  - Spesso utilizziamo strutture dati che rappresentano implementazioni concrete di alberi.
- Questo tipo di ADT lo incontriamo nella vita di tutti i giorni:
  - L'albero genealogico della propria famiglia (da cui deriva la maggior parte della terminologia impiegata nella teoria degli alberi).
  - Nei tornei sportivi.
  - Per rappresentare l'organigramma di aziende.
  - Per rappresentare l'analisi sintattica dei linguaggi di programmazione.
  - Il file system di un sistema operativo.
  - Gerarchie
  - ...

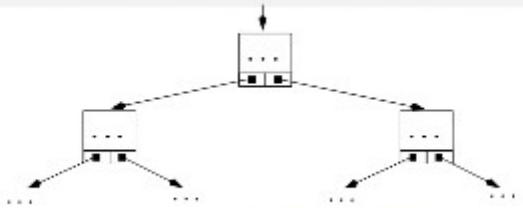
Alberi (Teoria dei grafi)

In teoria dei grafi un albero è un grafo non orientato nel quale due vertici qualsiasi sono connessi da uno e un solo cammino

- Definizione: Un albero è un insieme non vuoto di vertici ed archi (grafo) che soddisfa alcune proprietà:
  - Un vertice (o nodo) è un oggetto semplice che può essere dotato di un nome, e di una informazione associata (denominata spesso chiave o key).
  - Un arco è una connessione tra due nodi.
  - Un grafo non orientato, connesso e privo di cicli



Alberi binari



- (Nella sua versione più semplice) una albero binario `t` di oggetti di tipo `T` è definito come segue:
  - `t` è un puntatore **`NULL`** (albero vuoto) oppure
  - `t` è un puntatore ad un nodo (**`struct`**) contenente:
    - un campo `value` di tipo `T`
    - due campi `left` e `right` di tipo albero

```
struct node;
typedef node * albero;
struct node { T value;  albero left, right; };
```

Un albero binario è una struttura dati dinamica.

Alberi binari: terminologia

- I sottoalberi (possibilmente vuoti) di un nodo `N` sono detti sottoalbero sinistro e sottoalbero destro di `N`
- Se un nodo `N` punta nell'ordine a due (eventuali) nodi `N1`, `N2`
  - `N1` e `N2` sono detti rispettivamente figlio sinistro e figlio destro di `N`
  - `N` è detto nodo padre di `N1` e `N2`
- In un albero binario ci possono essere tre tipi di nodi:
  - Il nodo radice, che non ha padre
  - I nodi foglia, che non hanno figli
  - I nodi intermedi, che hanno padre e almeno un figlio

Alberi binari: terminologia - II

- Una catena di nodi dalla radice a una foglia è detta ramo
  - Il numero di nodi in un ramo è detto lunghezza del ramo
  - La massima lunghezza di un ramo è detta altezza dell'albero
  - L'altezza di un albero binario di N elementi è $h \in [\lceil log_2(N+1) \rceil, N]$
- Un albero binario di N elementi è bilanciato se la sua altezza è $h = \lceil log_2(N+1) \rceil$
  $\Longrightarrow$ tutti i rami hanno lunghezza `h` o `h-1`
- Un albero binario di N elementi è completo se la sua altezza è tale che $N = 2^h - 1$
  $\Longrightarrow$ tutti i rami hanno lunghezza `h`

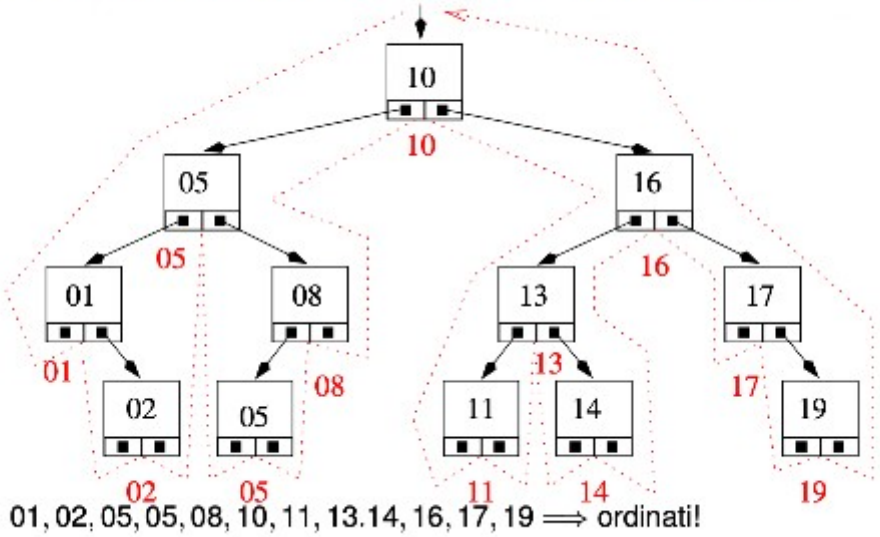
Albero di ricerca binaria

- Un albero di ricerca binaria è una struttura dati utile a mantenere dati ordinati.
- Assumiamo una relazione di ordine totale di precedenza "$\preceq$" tra gli elementi `T`
  - Es: ordine numerico, ordine alfabetico del campo "cognome", ecc.
- Un albero binario è un albero di ricerca binaria se ogni nodo `N` dell'albero verifica la seguente proprietà:
  - Tutti i nodi del sottoalbero di sinistra precedono strettamente `N`
  - Tutti i nodi del sottoalbero di destra sono preceduti da `N`

  (è possibile invertire lo "strettamente" tra sinistra e destra)

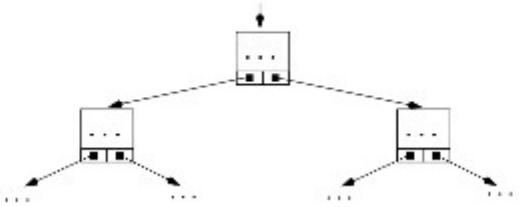
Nota: in alcuni casi non è previsto che ci possano essere due valori uguali nel valore valutato dalla relazione di precedenza (valore chiave)

Esempio: Visita ordinata di un albero di ricerca binaria



- Visita: 01, 02, 05, 05, 08, 10, 11, 13.14, 16, 17, 19 $\Longrightarrow$ ordinati!

Implementazione di un albero di ricerca binaria

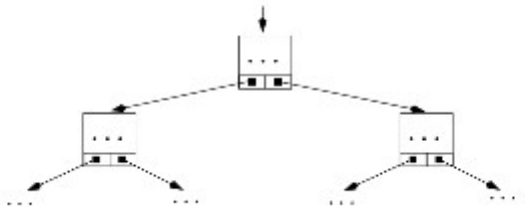


- Dati: un albero di ricerca binaria `t`
  - `t` punta al primo elemento inserito nell'albero (inizialmente **`NULL`**)
  - albero vuoto: `t==NULL`
  - albero pieno: `out of memory`

  $\Longrightarrow$ numero di elementi contenuti nell'albero limitato solo dalla memoria

N.B.: allocati solo gli `n` nodi necessari a contenere gli elementi

Implementazione di un albero di ricerca binaria II



- Funzionalità:
  - `init`: pone `t=NULL`
  - `search` (cerca un elemento `val` in `t`):
    1. se `t == NULL`, restituisce `NULL`
    2. se `val == t->value`, restituisce `t`
    3. se `val < t->value`, cerca ricorsivamente in `t->left`
    4. se `val > t->value`, cerca ricorsivamente in `t->right`
  - ...
  - `insert` (inserisce un elemento `val` in `t`):
    1. se `t` è vuoto, `t == NULL`:
       - crea un nuovo nodo per il puntatore `tmp`
       - pone `tmp->value=val`, `tmp->left=NULL`, `tmp->right=NULL`,
       - pone `t=tmp`
    2. se `val < t->value`, inserisci ricorsivamente in `t->left`
    3. se `val >=t->value`, inserisci ricorsivamente in `t->right`
  - `print` (stampa in modo ordinato l'albero `t`): Se l'albero non è vuoto
    - stampa ricorsivamente il sottoalbero sinistro `t->left`
    - stampa il contenuto del nodo puntato da `t`: `t->value`
    - stampa ricorsivamente il sottoalbero destro `t->right`
  - `deinit`: se l'albero non è vuoto:
    - applica ricorsivemente `deinit` ai sottoalberi sinistro `t->left` e destro `t->right`
    - applica **`delete`** al nodo puntato da `t`
  - [`remove` non analizzata qui]

Grafi

Debugger

g++ **-g** -> debugging
gdb a.out -> entro in debugger
g++ -Wall -> per dire al compilatore di segnalare ogni warning.

Valore più piccolo possibile per intero

#include <climits>
INT-MIN